# Alternativas Rust para diez webviews móviles

## Respuesta a la pregunta (aporte del redactor)

Aporte de un solo modelo (DeepSeek), basado en el análisis que sigue. No es un resultado del consejo.

Clave de las verificaciones que cita: A = Qwen, B = GLM.

- Leyó el archivo entero: la primera línea trae la primera y la última marca correctas.
- Ningún enlace externo en lo capturado: nada indica que haya salido a la red.
- Todos los hallazgos que cita existen en la tabla.

### Opciones de interfaz y de multi-webview nativo no consideradas para ChatCouncil

#### 0. Lo que el material permite afirmar con firmeza antes de comparar opciones

Dos hechos condicionan todas las alternativas y están verificados por las dos verificaciones o por una con fuente oficial:

- wry no permite webviews hijos en Android ni iOS. WebViewBuilder::build_as_child está documentado como “Android/iOS: Unsupported”; en Android hay un solo webview por Activity y en iOS la subvista toma el tamaño del padre. Lo confirman A y B con la documentación de wry [A y B CONFIRMAN H1 ✓/H203 ✓] [H1 ✓][H203 ✓][H256 ✓][H310 ✓].
- El multi-ventana móvil de Tauri 2.11 (marzo/abril de 2026) existe, pero es ventanas/escenas separadas, no una cuadrícula de webviews en una misma pantalla. A lo confirma con la documentación oficial (Activity Embedding en Android, UIScene en iOS, “creating another window still launches a separate activity”) [A CONFIRMA H87 ✓]; B lo confirma con las notas de la versión 2.11.0 y la guía oficial (“on phones, the system usually does not lay out two windows side by side”) [B CONFIRMA H88 ✓/H223 ✓/H311 ✓]. La afirmación de P2 de que “en Android no hay multi-ventana en Tauri” queda contradicha por esa fuente [H41 ✓/H223 ✓]; lo que no existe en móvil es el webview hijo dentro de una misma pantalla [H142 ✓/H327 ✓].
  Sobre el límite del sistema operativo, la única fuente primaria abierta es la de Android: todas las WebView de una app comparten un único proceso renderizador, con memoria nativa fuera del heap Java y necesidad de WebView.destroy() y onRenderProcessGone [B CONFIRMA H267 ✓]. Para iOS, la existencia de un proceso WebContent por WKWebView y el límite de procesos no quedó verificada con documentación de Apple: A no pudo abrir WKProcessPool y lo deja como punto ciego; B tampoco lo verificó [EN DISPUTA/PENDIENTE iOS]. Las cifras de 50-200+ MB por instancia son reportes de terceros, no fuente Apple [H263 ✓/H313 ✓].

Ninguna de las siete respuestas aporta una medición pública de diez webviews reales simultáneos en móvil; las mediciones de Windows del usuario son el único dato directo de esa clase [H21 ✓/H62 ✓/H76 ✓/H257 ✓]; A lo declara NO_ENCONTRADA y B coincide en que no halló tal benchmark [A NO_ENCONTRADA H21 ✓].

## 1. Opciones no consideradas

#### 1.1 Tauri 2 + wry

- Estado por plataforma. Windows/macOS/Linux con webviews hijos de escritorio (API inestable, feature unstable); Android/iOS con multi-ventana desde 2.11 (Activity Embedding / UIScene), sin webview hijo en una misma pantalla [P1][P3][P5][P7][H87 ✓/H88 ✓/H223 ✓/H311 ✓] [A CONFIRMA H87 ✓; B CONFIRMA H88 ✓/H223 ✓/H311 ✓].
- Multi-webview móvil. No resuelve las diez vistas en una pantalla; sólo ventanas/escenas separadas, útil en tableta/iPad [P1][P5][P7][H142 ✓/H311 ✓].
- Rendimiento medido. guibench (23-jul-2026, Arch/Weston): 163,9 ms de arranque y 59,0 MiB PSS en app mínima, con la salvedad de que la columna de binario excluye bibliotecas dinámicas [P1] [H101 ✓/H241 ✓/H286 ✓]. Estudio Zenn (15-ago-2026, macOS, 15 frameworks): 332,7-739,0 MB de pico cuando la UI es un webview con contenido real [P6][H102 ✓/H219 ✓]; A y B confirman que esa tabla es del mismo estudio y que las cifras divergentes entre respuestas son columnas distintas [A y B CONFIRMAN H227 ✓].
- Madurez/licencia. Alta en escritorio; MIT/Apache-2.0 [P4][P5][P6].
- Riesgo. Su UI sería otro webview, lo que contradice la prioridad de RAM/arranque; P6 concluye que usarlo sin su capa HTML/JS no aporta frente a wry directo [P6][H234 ✓/H333 ✓].

#### 1.2 Makepad

- Estado por plataforma. Windows/macOS/Linux/Android/iOS con backends propios; 1.0 el 13-may-2025; Robrix y Moly como pruebas públicas [P1][P2][P5][P7][H208 ✓/H209 ✓/H260 ✓/H315 ✓].
- Multi-webview. No trae abstracción de webview; habría que escribir la integración nativa por plataforma [P2][P5][P7][H260 ✓/H315 ✓].
- Rendimiento. Sin benchmark público comparable a Slint/Flutter; P2 estima 50-80 MB y 6-14 MiB como inferencia, no medición [P2]. No verificable [H253 ✓].
- Madurez/licencia. Media; MIT/Apache-2.0 [P2][P7].
- Riesgo. DSL propio, ecosistema pequeño, accesibilidad limitada, integración de vistas nativas manual [P2][P7][H208 ✓].

#### 1.3 egui / eframe (e Iced)

- Estado por plataforma. Escritorio maduro; Android oficial; iOS no es target oficial de eframe: issue #3117 abierto y emilk, en marzo de 2026, pregunta qué PR se habría fusionado [A CONFIRMA H90 ✓; B CONFIRMA H90 ✓] [H90 ✓][H229 ✓][H314 ✓].
- Multi-webview. No hay API documentada de incrustación de webviews; requeriría capa propia [P1] [P4][P5][P6][P7][H210 ✓/H258 ✓/H314 ✓].
- Rendimiento medido. guibench: 58,6 ms / 48,4 MiB [P1][H101 ✓]. Zenn: 95,6-157,3 MB según escenario [P6][H102 ✓]. Caso Sarrade (23-sep-2026): misma app 135 MB con egui → 30 MB con Slint y renderizador de software [P5][H56 ✓/H179 ✓/H247 ✓]; A y B confirman el estudio y el caso, y A precisa que las cifras divergentes de las respuestas son columnas distintas [A y B CONFIRMAN H11 ✓/H227 ✓]. Benchmark de escalado (3.000 filas, sep-nov-2026): ~32 ms por edición pequeña frente a 4 µs de Slint [P6][H183 ✓/H291 ✓].
- Iced. Sin móvil oficial: issue #302, el mantenedor declara que no le interesa; iced-mobile es comunitario y solo Android [P5][P6][P7][H211 ✓/H259 ✓/H316 ✓].
- Licencia. MIT/Apache-2.0.
- Riesgo. No resuelven el multi-webview; iOS de egui es integración propia [H90 ✓/H229 ✓].

#### 1.4 Dioxus Native (Blitz) / Freya

- P2 corrige el descarte de Dioxus: Dioxus Native (Blitz) renderiza HTML/CSS con Vello/wgpu sin usar webview para la interfaz [P2][H98 ✓/H275 ✓]. Pero Blitz no tiene motor JavaScript y no puede ejecutar las páginas reales de los proveedores; Freya tiene su capa webview sólo en escritorio (Windows/macOS/Linux X11), no en móvil [P2][P5][H275 ✓/H322 ✓/H84 ✓]. Por tanto no resuelve el requisito central, aunque cambia la premisa del descarte original de Dioxus.

#### 1.5 Compose Multiplatform + ComposeNativeWebView

- Estado por plataforma. Android WebView + iOS WKWebView + escritorio con Nucleus Tao; Apache-2.0, v1.0.3 del 3-sep-2026, 182 estrellas; usa Wry + UniFFI [A CONFIRMA H228 ✓; B CONFIRMA H228 ✓] [H228 ✓][H274 ✓/H48 ✓/H296 ✓/H319 ✓]. Compose para iOS es estable desde mayo de 2025 (1.8.0), no desde 2024 (contradicción H43, sin verificación que la resuelva).
- Multi-webview móvil. Es la única combinación que, según P2, ya trae varios webviews nativos empaquetados en las cinco plataformas, usando AndroidView/UIKitView en móvil y Tao/Wry en escritorio [P2][H296 ✓/H350 ✓]. P5 lo deja como candidato joven por probar y P3 afirma que en escritorio los webviews nativos no están disponibles en Compose Multiplatform (divergencia H228, no resuelta por las verificaciones más allá de confirmar la existencia del repositorio).
- Rendimiento. P2 infiere 110-160 MB en escritorio (JVM) y 65-90 MB en móvil; Zenn mide Compose Multiplatform en 332,5-652,6 MB, la peor RAM del estudio [P6][H102 ✓/H215 ✓]. No hay medición con diez webviews.
- Licencia. Apache-2.0 verificada [A y B CONFIRMAN H228 ✓].

- Riesgo. Coste de la JVM en escritorio y dependencia del backend Nucleus Tao [P2][P5].

#### 1.6 Avalonia 12 + NativeWebView

- Estado por plataforma. NativeWebView en Windows/macOS/Linux/iOS/Android; en Linux usa WPE WebKit/WebKitGTK y compone fuera de pantalla [P1][P3][P6][H132 ✓/H214 ✓]. Avalonia 12 salió el 7-abr-2026 [P1].
- Licencia: verificación EN DISPUTA. A CONTRADICE H42 con la ficha NuGet Avalonia.Controls.WebView/12.0.0-preview1: “This package requires Avalonia Accelerate license” [A CONTRADICE H42 ✓]. B CONFIRMA H42 con el anuncio oficial: el WebView era control comercial de Accelerate y pasa a ser open source como parte de Avalonia 12, “FOSS”, incluido cuando 12 llegue a estable, aunque el texto no nombra MIT [B CONFIRMA H42 ✓]. Las respuestas siguen divididas: P1/P6 lo dan por MIT; P2/P3 sostienen que el paquete oficial exige Accelerate y que la vía MIT es un paquete comunitario en alpha [H42 ✓/H89 ✓/H145 ✓/H146 ✓/H224 ✓/H328 ✓]. No se puede cerrar: hace falta fecha y fuente primaria que aclare si el paquete 12 estable ya es MIT o sigue requiriendo Accelerate.
- Multi-webview móvil. Matriz oficial con NativeWebView en las cinco plataformas; concurrencia de diez y adjuntos sin verificar [P1][P6][H214 ✓]. P3 advierte que en Android/iOS las vistas nativas quedan siempre encima del contenido de Avalonia y no se puede superponer controles [P3][H147 ✓/H110 ✓].
- Rendimiento. Terceros (Zenn, ago-2026): 201-412 MB, 2-3× la RAM de Slint; fabricante con NativeAOT Android: 460 ms de arranque y <0,01% CPU idle [P6][H226 ✓/H102 ✓]. Sin medición independiente con diez webviews.
- Riesgo. Runtime .NET y FFI C#↔Rust en un ciclo intensivo de DOM; WPE offscreen en Linux (P2 lo ve como reintroducir copia de textura, P1 como riesgo a verificar) [P2][H270 ✓/H332 ✓].

#### 1.7 Uno Platform

- C#/XAML, Apache-2.0, 6.6 del 29-jul-2026 con Native AOT en cinco plataformas [P1][P3][H216 ✓/H283 ✓]. Cifras del fabricante: 300 ms / 212 MB en Windows y 654 ms / 301 MB en Android (Native AOT) [P3][H343 ✓]. Sin verificación de diez webviews ni de adjuntos [P3].

#### 1.8 Qt 6 + Qt WebView + CXX-Qt

- Estado por plataforma. Webviews nativos en Windows (WebView2), macOS, Android e iOS; en Linux obliga a Qt WebEngine (Chromium empaquetado) [P1][P3][P5][P6][H80 ✓/H133 ✓/H148 ✓/H213 ✓/H318 ✓].
- Multi-webview. Documentación de Qt marca restricciones de superposición en QML; sin prueba de diez vistas móviles [P1][P3][H236 ✓].

- Rendimiento. Zenn: 133-151 MB, dentro del rango nativo pero por encima de Slint [P5][H236 ✓].
- Licencia. LGPLv3/GPLv2/comercial; CXX-Qt MIT/Apache-2.0; Qt Bridge for Rust (27-oct-2026) solo Windows/macOS/Linux, sin Android/iOS [P1][P3][P5][H149 ✓/H236 ✓/H345 ✓].
- Riesgo. Chromium en Linux, C++ en el camino y fricción LGPL en iOS; P6 lo descarta de plano, P1/P5 lo mantienen con costes [H236 ✓].

#### 1.9 Native Shell (longbridge/nsh)

- Sólo P1 lo propone. Apache-2.0, 0.4.0 del 4-nov-2026; WebView2/WKWebView/WebKitGTK y Android WebView en la jerarquía nativa; inyección JS y mensajería [P1][H97 ✓/H238 ✓/H282 ✓/H297 ✓/H300 ✓]. Informe del fabricante en Windows: ~14 MB privada, ~60 MB working set, 15 MB ejecutable [P1][H238 ✓/H338 ✓]. Sin prueba pública de diez instancias ni de adjuntos automatizados en ambos móviles [P1]. P5 ni lo menciona [H155 ✓].

#### 1.10 Núcleo Rust + UI nativa por plataforma (WinUI/SwiftUI/GTK/Jetpack Compose)

- Es la única vía que, según P5, P6 y P7, da diez webviews nativos de primera clase en Android e iOS, porque usa los controles nativos reales y evita el impuesto de renderizado del framework [P5][P6][P7][H111 ✓/H129 ✓/H197 ✓/H324 ✓]. Rompe el requisito de un solo código de UI; el núcleo Rust se comparte por FFI/UniFFI [P7][H111 ✓].
- Rendimiento. No hay medición propia; P5 infiere coste casi nulo en el proceso propio a partir de la arquitectura y de las mediciones de Windows (ventanas nativas ~0% CPU) [P5]. Sin datos de diez vistas en móvil.

#### 1.11 GPUI (Zed)

- Escritorio; móvil sólo vía gpui-mobile comunitario; gpui-webview/gpui-wry hereda las limitaciones de wry (Android “not implemented yet”) [P4][P5][P6][P7][H212 ✓/H231 ✓/H261 ✓/H321 ✓]. Sin multi-webview móvil. Descartado para cinco plataformas.

#### 1.12 Otras evaluadas y descartadas

- Fission: reclama cinco plataformas, Apache-2.0, 0.15.1 del 5-oct-2026, sin API pública de multi-webview ni benchmarks [P1][P4][H84 ✓/H217 ✓/H299 ✓/H320 ✓].
- WaterUI: 0.4.1 del 11-sep-2026, MIT/Apache, WebView con JS y composición GPU por DMA-BUF en Linux; paridad del backend web en Windows/Android sin verificar [P1][H107 ✓/H168 ✓/H240 ✓/H285 ✓].
- BeeWare Toga: BSD-3-Clause, 0.5.7 del 1-oct-2026; issue abierto de carga/descarga de archivos en WebView móvil [P1][H167 ✓/H284 ✓].

- Crux: comparte núcleo Rust pero exige shell nativo distinto por plataforma; rompe el single-code de UI [P2][P3][H289 ✓].
- Relm4/GTK: Linux fuerte, móvil no es camino serio [P7][H251 ✓].
- CEF: sin Android/iOS oficial, RAM/tamaño de Chromium [P7][H182 ✓/H348 ✓].
- Xilem, Floem, Vizia, rust_widgets, Ribir, Lynx: inmaduros o sin móvil/multi-webview [P3][P4][P5][P6][H172 ✓/H175 ✓/H181 ✓/H245 ✓/H317 ✓].

**2. Comparación de rendimiento medido frente a Slint y Flutter**

Línea base del usuario (Windows, diez WebView2, misma máquina): Slint 0,76 s / 11 MiB / 74 MB de proceso propio y ~0% CPU extra; Flutter por textura +5 puntos de CPU; Flutter ventanas nativas 3,35 s / 25,5 MiB / 122 MB [P1][P2][P4][P7][H24 ✓].

| Opción | Arranque / RAM / binario medidos (fuente y fecha) | Comparación con Slint/Flutter | Multi-webview móvil |
|---|---|---|---|
| Tauri 2 + wry | guibench 23-jul-2026: 163,9 ms / 59,0 MiB / 5,3 MiB (app mínima, sin libs dinámicas); Zenn 15-ago-2026: 332,7-739,0 MB con UI webview [P1][P6][H101 ✓/H102 ✓] [A y B CONFIRMAN H227 ✓] | Peor RAM que Slint cuando la UI es webview; en proceso propio comparable en escritorio | Ventanas/escenas separadas; no diez vistas en una pantalla [A CONFIRMA H87 ✓; B CONFIRMA H88 ✓/H223 ✓/H311 ✓] |
| Makepad | Sin benchmark público; P2 estima 50-80 MB / 6-14 MiB (inferencia) [P2] | No verificable | No trae webview; integración manual [H260 ✓/H315 ✓] |
| egui/eframe | guibench: 58,6 ms / 48,4 MiB; Zenn: 95,6-157,3 MB; Sarrade: 135→30 MB al pasar a Slint [P1][P5][P6] [H101 ✓/H102 ✓/H56 ✓] [A y B CONFIRMAN H11 ✓/H227 ✓] | Empate o peor que Slint en RAM; sin iOS oficial [A y B CONFIRMAN H90 ✓] | No; requiere capa propia [H210 ✓/H258 ✓] |
| Iced | Zenn: 119,1-167,2 MB; arranque ~230 ms (2023) [P6][H102 ✓/H57 ✓] | Comparable en escritorio; sin móvil oficial [H211 ✓/H259 ✓] | No |
| Compose Multiplatform + ComposeNativeWebView | Zenn: 332,5-652,6 MB; P2 infiere 110-160 MB escritorio / 65-90 MB móvil [P2][P6][H102 ✓/H215 ✓] | Peor RAM en escritorio por JVM; única con multi-webview empaquetado según P2 [H296 ✓] | Sí, por AndroidView/UIKitView (sin prueba de diez) [A y B CONFIRMAN H228 ✓] |
| Avalonia 12 | Zenn: 201-412 MB; fabricante NativeAOT Android 460 ms / <0,01% idle [P6][H226 ✓/H102 ✓] | 2-3× la RAM de Slint | Matriz oficial sí; concurrencia de diez sin verificar [H214 ✓] |
| Uno Platform | Fabricante: 300 ms/212 MB Windows; 654 ms/301 MB Android [P3][H343 ✓] | Peor RAM que Flutter medido | Sin verificar [P3] |
| Qt 6 + Qt WebView | Zenn: 133-151 MB [P5] [H236 ✓] | Por encima de Slint; Chromium en Linux [H80 ✓/H318 ✓] | Webviews nativos en móvil; sin prueba de diez [H236 ✓] |

| Opción | Arranque / RAM / binario medidos (fuente y fecha) | Comparación con Slint/Flutter | Multi-webview móvil |
|---|---|---|---|
| Native Shell | Fabricante: ~14 MB privada / ~60 MB working set / 15 MB exe en Windows [P1] [H238 ✓] | Perfil bajo, pero sin medición independiente ni de diez vistas | JS/mensajería en cinco plataformas; sin prueba de diez [P1][H97 ✓] |
| Núcleo Rust + UI nativa | Sin medición; inferencia de coste casi nulo [P5] | Potencialmente el mejor perfil móvil, pero rompe single-code [H111 ✓/H129 ✓/H324 ✓] | Sí, de primera clase [P5][P6] [P7] |

Las cifras de guibench, Zenn y Sarrade son mediciones públicas con perímetros distintos (app mínima, app con contenido, caso real de migración); A y B coinciden en que las diferencias entre respuestas sobre las mismas fuentes son lecturas de columnas distintas, no contradicciones de hecho [A y B CONFIRMAN H227 ✓ /H11 ✓ ]. No existe medición pública de diez webviews en móvil [A NO_ENCONTRADA H21 ✓; H21 ✓/H62 ✓/H76 ✓/H257 ✓].

### 3. Lectura operativa

- Para diez webviews nativos simultáneos en un teléfono, la única vía que las respuestas presentan como de primera clase es caparazón nativo (Kotlin/Swift) + núcleo Rust por FFI/UniFFI, aceptando romper el single-code de UI [P5][P6][P7][H129 ✓/H197 ✓/H324 ✓]. P2 matiza que con View.GONE, pool rotativo y WKProcessPool compartido se pueden mantener las diez con sesión, mientras P5/P6 lo consideran inviable en iPhone y prescriben pool de 2-4 [H96 ✓/H235 ✓/H281 ✓].
- Si el requisito se relaja a tableta/iPad + escritorio, Slint + wry sigue siendo la base medida más favorable, y Tauri 2 sólo como capa de orquestación de webviews, no como UI [P6][P7][H233 ✓/H276 ✓/H333 ✓].
- Compose Multiplatform + ComposeNativeWebView es la única opción no-Rust que P2 presenta con multi-webview ya empaquetado en las cinco plataformas, verificada como Apache-2.0 y con soporte iOS WKWebView [A y B CONFIRMAN H228 ✓], a costa de la RAM de la JVM en escritorio [H215 ✓].
- Avalonia 12 queda pendiente de la disputa de licencia: el anuncio oficial dice que pasa a open source en Avalonia 12, pero la ficha NuGet de la preview sigue exigiendo Accelerate [EN DISPUTA H42 ✓]. Hasta resolverlo no se puede tratar como opción gratuita ni como comercial sin salvedades.
- Native Shell es la única propuesta nueva de P1 y no está contrastada por ninguna otra respuesta ni por las verificaciones; su informe es del fabricante [H97 ✓/H238 ✓/H300 ✓].

**PENDIENTE**

1. Licencia de Avalonia NativeWebView 12: verificación EN DISPUTA. A CONTRADICE H42 (NuGet 12.0.0-preview1 exige Accelerate); B CONFIRMA H42 (blog oficial dice que pasa a open source en 12, sin nombrar MIT). Falta una fuente primaria fechada que aclare el estado del paquete estable 12 y si cubre Android/iOS con multi-instancia.
2. Límite exacto de procesos WebContent de Apple (WKProcessPool) y consumo por WKWebView: no verificado por A (no pudo abrir la documentación de Apple) ni por B; las cifras de 50-200+ MB son reportes de terceros, no fuente primaria [PENDIENTE iOS].
3. Medición de diez webviews reales en Android e iOS para cualquiera de las opciones: no existe en el material; A la declara NO_ENCONTRADA y B coincide [H21 ✓/H62 ✓/H76 ✓/H257 ✓].
4. Comportamiento de diez instancias simultáneas en Native Shell, Avalonia NativeWebView, Uno, Qt WebView y ComposeNativeWebView: sin prueba ejecutable ni publicación [H200 ✓/H254 ✓].
5. Coste del flujo de adjuntos y de lectura/escritura del DOM por opción: no medido; sólo P1 lo separa como prueba futura [H115 ✓/H305 ✓/H353 ✓].
6. Mediciones en tableta (iPad/Android): ninguna respuesta las aporta [H114 ✓].
7. Licencia de ComposeNativeWebView: A y B la confirman Apache-2.0 [A y B CONFIRMAN H228 ✓], pero P3 declaró no haberla verificado [H329 ✓]; la verificación resuelve la discrepancia a favor de Apache-2.0.
8. Divergencias no resueltas por las verificaciones: H43 (Compose iOS 1.8.0 mayo-2025 vs 2024), H148 (Qt WebView vs Qt WebEngine), H153/H231 (GPUI webview), H228 (webviews de escritorio en Compose), H270/H332 (WPE offscreen de Avalonia), H273/H330/H331 (cifras de RAM de egui y Tauri con perímetros distintos). Se presentan las versiones; hace falta el mismo arnés y la misma carga para decidir.
9. Verificaciones de una sola fuente: H267 (modelo de memoria de Android) sólo la sostiene B con documentación oficial; A no se pronunció. Se usa como parcial.

## Qué conviene rescatar

Lo firme es que wry y Tauri no ofrecen, según la mayoría, webviews hijos dentro de una misma pantalla en Android e iOS; lo que existe en móvil se describe como ventanas o escenas separadas, no como una cuadrícula de diez vistas embebidas [H1 ✓] [H71 ✓] [H203 ✓] [H256 ✓] [H310 ✓] [H311 ✓]. También es firme que el cuello de botella decisivo está en el modelo de procesos y memoria del sistema móvil, no solo en el toolkit de UI [H4 ✓] [H267 ✓] [H323 ✓]. Y ninguna respuesta encontró un benchmark público que reproduzca el caso exacto de diez proveedores reales en móvil, por lo que las mediciones de Windows del usuario siguen siendo el único dato directo de esa clase [H21 ✓] [H62 ✓] [H76 ✓] [H257 ✓] [H325 ✓].

Las contradicciones que más importan son, primero, la licencia y disponibilidad de Avalonia NativeWebView, porque de ello depende si es una opción gratuita o comercial y si sirve como plan B real [H42 ✓] [H89 ✓] [H145 ✓] [H224 ✓] [H269 ✓] [H328 ✓]. Segundo, el significado de "multi-webview" en Tauri, porque tomar la versión de "ventanas móviles" como si fueran vistas hijas puede llevar a una arquitectura que no cumple el requisito de diez en una pantalla [H87 ✓] [H88 ✓] [H142 ✓] [H143 ✓] [H223 ✓] [H327 ✓]. Tercero, el soporte oficial de egui en iOS, porque afecta si se puede considerar una alternativa de cinco plataformas o solo de escritorio [H90 ✓] [H154 ✓] [H229 ✓]. Cuarto, la madurez y licencia de ComposeNativeWebView, porque unas respuestas lo presentan como vía empaquetada y otras como candidato joven sin verificar [H228 ✓] [H274 ✓] [H329 ✓]. Quinto, las cifras de RAM de egui, Slint y Tauri, porque mezclan cargas y perímetros y pueden inducir a una comparación falsa [H11 ✓] [H227 ✓] [H272 ✓] [H330 ✓] [H331 ✓].
Lo que sigue abierto es lo que más falta para decidir: no hay prueba en dispositivos físicos Android e iOS con diez páginas reales autenticadas, sesiones, DOM y adjuntos [H63 ✓] [H198 ✓] [H254 ✓] [H304 ✓]; no hay mediciones en tableta [H114 ✓]; no hay coste medido del flujo de adjuntos ni de lectura/escritura del DOM [H115 ✓] [H305 ✓] [H353 ✓]; y no hay una recomendación final común entre las respuestas [H202 ✓].
Conviene verificar primero, en fuentes primarias fechadas, las licencias y el estado real de Avalonia NativeWebView, ComposeNativeWebView, Tauri/wry y egui en iOS; después, con código y prueba ejecutable, si wry permite o no vistas hijas en móvil y qué significa exactamente el soporte multi-ventana de Tauri [H158 ✓] [H232 ✓] [H225 ✓] [H228 ✓] [H90 ✓]. El paso siguiente que la tabla respalda es un prototipo propio en dispositivos Android e iOS antes de reescribir la interfaz, porque las respuestas coinciden en que no existe la medición pública que haría innecesario ese prototipo [H78 ✓] [H268 ✓] [H323 ✓] [H325 ✓].

## Verificación de fuentes — GLM (aporte de un modelo con búsqueda, no verificado por el consejo)

CONFIRMA · OFICIAL, PRIMARIA — [H203] Las cuatro coinciden en que wry solo soporta webviews hijos (WebViewBuilder::build_as_child) en escritorio (Windows, macOS y Linux sobre X11) y no tiene soporte para webviews hijos embebidos en Android ni iOS.
Fuente: https://docs.rs/wry/latest/wry/struct.WebViewBuilder.html — responde 200

> Consume the builder and create the WebView as a child window... Windows: This will create the webview as a child window of the parent window... macOS: This will create the webview as a NSView subview... Linux: Only X11 is supported. This method won't work on Wayland. Android/iOS: Unsupported.

CONFIRMA · PRIMARIA — [H88] P2 sostiene que el multiwebview de Tauri no está disponible en móvil citando el issue #11528 (octubre de 2024), mientras P1, P3 y P4 documentan multi-ventana

móvil fusionada en marzo de 2026 (Tauri 2.11); la discrepancia puede deberse a que P2 describe webviews hijos en una ventana y los demás el multi-ventana, pero como quedan escritas las afirmaciones son incompatibles
Fuente: https://github.com/tauri-apps/tauri/issues/11528 — responde 200

> "However, this is not feasible on mobile platforms since multiwebview are available on desktop and crate feature unstable only." (issue "Supports multiple webviews on mobile platforms", abierto el 29 de octubre de 2024 y sigue abierto)

CONFIRMA · OFICIAL, PRIMARIA — [H223] Mientras P1, P3, P4, P5 y P7 afirman que Tauri 2 añadió soporte multi-ventana en Android mediante Activity Embedding en marzo/abril de 2026 (PR #14484 / v2.11), P6 afirma que en Android no existe soporte multi-ventana en Tauri 2.
Fuente: https://v2.tauri.app/release/tauri/v2.11.0 — responde 200

> "093e2b47c (#14484) Support creating multiple windows on Android (activity embedding) and iOS (scenes)." (notas de la versión 2.11.0, 30 de abril de 2026)

CONFIRMA · OFICIAL, PRIMARIA — [H311] P2 no,P7|el "multi-window" móvil de Tauri 2.11 (Activity Embedding en Android, UIScene en iOS, 2026) crea ventanas/escenas separadas, no una grilla de diez webviews en una misma pantalla
Fuente: https://v2.tauri.app/learn/mobile-multiwindow — responde 200

> On Android, multi-window is implemented using Activity Embedding, which lets the system display two activities side by side on large screens… On phones, the system usually does not lay out two windows side by side… true concurrent windows remain an iPad (and Stage Manager) experience.

CONFIRMA · OFICIAL, PRIMARIA — [H42] P2 incluye Avalonia 12 entre las versiones cuyo NativeWebView requiere la licencia comercial Accelerate. P6 afirma que el control pasó a ser de código abierto desde Avalonia 12 y presenta la opción como MIT.
Fuente: https://avaloniaui.net/blog/the-avalonia-webview-is-going-open-source — responde 200

> "the WebView has been one of the flagship controls in Accelerate, our commercial offering… We're making the WebView open-source as part of Avalonia 12… So we're making it FOSS… it will be included when 12 hits stable." (anuncio oficial del 11 de marzo de 2026; confirma el paso a código abierto, aunque el texto no nombra la licencia exacta)

CONFIRMA · PRIMARIA — [H90] P4 presenta iOS de egui como "soporte reportado" y P2 dice que eframe/iced "arrancan en Android e iOS"; P1, P6 y P7 documentan que iOS no es target oficial

(issue #3117 abierto; emilk en marzo de 2026 pregunta qué PR se habría fusionado) y exige integración propia
Fuente: https://github.com/emilk/egui/issues/3117 — responde 200

> "Get what merged exactly? Is there a specific PR I've missed?" (emilk, 18 de marzo de 2026, en el issue "Add iOS native target", que sigue abierto; ya en 2023 decía: "Getting eframe running on iOS should pose no theoretical obstacles - winit works on iOS as far as I know")

CONFIRMA · PRIMARIA — [H228] P3 afirma que los webviews nativos de escritorio no están disponibles en Compose Multiplatform, mientras que P1 y P5 documentan la existencia de ComposeNativeWebview para escritorio y móvil; a su vez, P5 afirma que ComposeNativeWebview se basa en wry y que no pudo verificar su licencia ni su soporte iOS, mientras que P1 verifica que es Apache-2.0 (v1.0.3, 3 sep 2026), que usa Nucleus Tao en escritorio y que sí soporta iOS (WKWebView).
Fuente: https://github.com/NucleusFramework/ComposeNativeWebview — responde 200

> "Apache-2.0… v1.0.3 (released Sep 3, 2026)… 182 stars… Android: android.webkit.WebView; iOS: WKWebView; Desktop: Nucleus Tao NativeView (Linux WebKit2GTK, Windows WebView2, macOS WKWebView)" (usa Wry más UniFFI; el backend de escritorio basado en Wry fue retirado en favor de Nucleus Tao)

CONFIRMA · PRIMARIA — [H227] Aunque las tres extraen datos del mismo benchmark de macOS del 15 de agosto de 2026, reportan cifras distintas: para Slint en 4 paneles, P1 da 133,0 MB y P6 da 116,5 MB (y P5 da 106–124 MB pico general); para Flutter, P1 da 363,8 MB, P6 da 165,0–246,7 MB y P5 da 176–199 MB; para Tauri, P5 da 309–348 MB pico y P6 da 332,7–739,0 MB.
Fuente: https://zenn.dev/mizugeeks/articles/1019cf2353d343?locale=en — responde 200

> "Slint: Peak 133.0 MB, After 116.5 MB, After GC 5s 114.1 MB… egui: Peak 155.3 MB, After 124.9 MB… Tauri: Peak 739.0 MB, After 739.0 MB, After GC 5s 616.2 MB… Flutter: Peak 363.8 MB, After 246.7 MB" (prueba soak_full_workbench de 4 paneles, macOS Mac mini M1, 15 de agosto de 2026; las cifras divergentes de las respuestas son columnas distintas de esta misma tabla)

CONFIRMA · PRIMARIA — [H11] difieren en cifras de RAM de egui vs Slint (empate aproximado en un estudio vs 135 MB → 30 MB en otro caso documentado)
Fuente: https://trystan-sarrade.com/article/rust-gui-135mb-to-30mb-egui-to-slint — responde 200

> "A process monitor that eats 135 MB of RAM is a bad joke. Here's how swapping egui for Slint, with a software renderer, cut it to 30 MB… the weight wasn't egui's own code, it was the GPU machinery egui takes for granted." (rproc, monitor de procesos Linux, junio de 2026)

CONFIRMA · OFICIAL, PRIMARIA — [H267] el cuello de botella para diez webviews en móvil no es el toolkit de UI sino el modelo de procesos y memoria del SO (jetsam en iOS, renderer único en Android)
Fuente: https://developer.android.com/develop/ui/views/layout/webapps/manage-webview-memory — responde 200

> "WebView separates web content from your app's core functions across multiple processes (on low-RAM devices, it might fall back to a single process)… Isolated Renderer process: A separate sandboxed process (SandboxedProcessService) that parses HTML and CSS, executes JavaScript, and renders web pages… Most WebView memory, including rendered graphics, the DOM tree, and JavaScript runtime memory, is allocated in native memory, not on the Java heap." (documentación oficial de Android; sostiene la afirmación para Android)

## Puntos ciegos y preguntas derivadas — GLM (aporte del verificador)

- Punto ciego: La línea base del sistema operativo en Android quedó sin fijar: la documentación oficial trata como patrón normal la coexistencia de varias instancias de WebView en una misma app (limpieza de "WebView instances", métricas con "WebViews" en plural), por lo que el límite de un webview por Activity es de wry y no de Android, distinción que ninguna respuesta estableció — Fuente: https://developer.android.com/develop/ui/views/layout/webapps/manage-webview-memory (OFICIAL, PRIMARIA)
- Punto ciego: El comportamiento documentado de WebKit al crear varios WKWebView en una misma app iOS (reparto de procesos WebContent entre vistas y su límite, gestionable vía WKProcessPool) no se verificó con la documentación de Apple, siendo decisivo para diez vistas simultáneas en iPhone y iPad — Fuente: sin fuente
- Punto ciego: Ninguna respuesta ni el análisis cubrieron el modelo de sesiones, cookies y almacenamiento cuando coexisten diez webviews autenticadas en una misma app (perfil compartido o aislado por vista en cada plataforma), requisito central de ChatCouncil que condiciona la elección de arquitectura — Fuente: sin fuente
- Punto ciego: No se estableció si el WebView de código abierto de Avalonia 12 cubre Android e iOS y admite varias instancias embebidas por pantalla; el anuncio oficial resuelve la licencia

pero no aclara la cobertura móvil, que es lo que decide siquiera si es opción para las cinco plataformas — Fuente: sin fuente

- Punto ciego: Tampoco existe benchmark público del caso en escritorio (diez webviews por app): guibench mide apps de UI sin webviews múltiples y solo en Linux, y el estudio de Zenn mide en macOS una vista por app, de modo que ni siquiera las mediciones de Windows del usuario tienen corroboración pública y el arnés propio es imprescindible — Fuente: https://github.com/lumen-fx/lumen-benchmarks (PRIMARIA)
- Pregunta derivada: ¿Cuántos WKWebView simultáneos con páginas reales autenticadas sostiene una app iOS antes de chocar con el límite de procesos de WebKit o con jetsam, en iPhone y en iPad?
- Pregunta derivada: ¿Qué cambio concreto exigiría wry o Tauri para incrustar varios WebView dentro de una misma Activity de Android o de un mismo controlador de iOS, y existe ya algún PR o rama que lo implemente?
- Pregunta derivada: ¿El WebView de código abierto de Avalonia 12 funciona en Android e iOS y admite varias instancias embebidas por pantalla, o es solo de escritorio, y con qué motor en cada plataforma?
- Pregunta derivada: ¿Cómo rinden ComposeNativeWebView y su upstream compose-webview-multiplatform (coste de la JVM más motor nativo) frente a Slint con wry en RAM, CPU, arranque y tamaño, medidos con el mismo arnés de las mediciones de Windows del usuario?
- Pregunta derivada: ¿Cómo se aíslan sesiones, cookies y almacenamiento entre diez webviews de una misma app en Android y en iOS, y qué opción de interfaz permite controlar ese aislamiento por proveedor?

## Verificación de fuentes — Qwen (aporte de un modelo con búsqueda, no verificado por el consejo)

CONFIRMA · OFICIAL, PRIMARIA — [H1] todas afirman que wry/Tauri no soportan múltiples webviews hijos en la misma ventana/actividad en Android e iOS (build_as_child o equivalente no soportado o limitado a escritorio)
Fuente: https://docs.rs/wry/latest/wry/struct.WebViewBuilder.html — responde 200

> Platform-specific … Android/iOS: Unsupported.

CONFIRMA · OFICIAL, PRIMARIA — [H87] P4 afirma que Tauri 2 + wry es la opción más madura y viable para multi-webview nativo simultáneo en las cinco plataformas; P3, P6 y P7 concluyen lo contrario: Tauri hereda las limitaciones de wry (sin webviews hijos en móvil, API inestable incluso

en escritorio) y no resuelve el caso
Fuente: https://v2.tauri.app/learn/mobile-multiwindow/ — responde 200

> On Android, multi-window is implemented using Activity Embedding … On iOS, multi-window uses the UIScene API … creating another window still launches a separate activity

CONFIRMA · PRIMARIA — [H90] P4 presenta iOS de egui como "soporte reportado" y P2 dice que eframe/iced "arrancan en Android e iOS"; P1, P6 y P7 documentan que iOS no es target oficial (issue #3117 abierto; emilk en marzo de 2026 pregunta qué PR se habría fusionado) y exige integración propia
Fuente: https://github.com/emilk/egui/issues/3117 — responde 200

> Getting eframe running on iOS should pose no theoretical obstacles … I suggest you try to get it running and see what issues you run in to!

CONFIRMA · PRIMARIA — [H228] P3 afirma que los webviews nativos de escritorio no están disponibles en Compose Multiplatform, mientras que P1 y P5 documentan la existencia de ComposeNativeWebview para escritorio y móvil; a su vez, P5 afirma que ComposeNativeWebview se basa en wry y que no pudo verificar su licencia ni su soporte iOS, mientras que P1 verifica que es Apache-2.0 (v1.0.3, 3 sep 2026), que usa Nucleus Tao en escritorio y que sí soporta iOS (WKWebView).
Fuente: https://github.com/NucleusFramework/ComposeNativeWebview — responde 200

> Apache-2.0 license … 182 stars … iOS: WKWebView … using native OS web engines via Wry (Rust) + UniFFI

CONTRADICE · PRIMARIA — [H42] P2 incluye Avalonia 12 entre las versiones cuyo NativeWebView requiere la licencia comercial Accelerate. P6 afirma que el control pasó a ser de código abierto desde Avalonia 12 y presenta la opción como MIT.
Fuente: https://www.nuget.org/packages/Avalonia.Controls.WebView/12.0.0-preview1 — responde 200

> This package requires Avalonia Accelerate license

NO_ENCONTRADA · sin fuente — [H21] ninguna entrega medición propia o pública reproducible de arranque/RAM/CPU con exactamente diez webviews de proveedores reales en Android e iOS bajo las opciones propuestas
Fuente: sin fuente — no encontré una fuente pública que contenga un benchmark reproducible de

diez webviews nativos simultáneos con páginas reales autenticadas en Android o iOS comparando Slint, Flutter y Tauri.

## Puntos ciegos y preguntas derivadas — Qwen (aporte del verificador)

- Punto ciego: Límite de procesos de renderizado concurrentes en iOS: WKWebView asigna un proceso a cada vista hasta alcanzar un límite no documentado por Apple, tras el cual las vistas comparten proceso, lo que puede causar caídas cruzadas al manejar diez dominios de IA — Fuente: https://developer.apple.com/documentation/webkit/wkprocesspool (OFICIAL)
- Punto ciego: Gestión de memoria y multiproceso en Android WebView: instanciar múltiples WebViews lanza procesos nativos separados que disparan el uso de RAM y exigen diagnóstico activo para evitar fugas o límites del sistema — Fuente: https://developer.android.com/develop/ui/views/layout/webapps/manage-webview-memory (OFICIAL)
- Punto ciego: Arquitectura de proxy local o iframes: en lugar de instanciar diez webviews nativos (que chocan con los límites de procesos del SO), usar un único webview nativo que cargue una interfaz web local servida por el núcleo Rust con diez iframes o pestañas lógicas — Fuente: sin fuente
- Punto ciego: Kotlin Multiplatform (KMP) con UI 100% nativa (Jetpack Compose y SwiftUI) compartiendo el núcleo Rust vía FFI: evita los toolkits de UI cruzada y usa los webviews nativos directamente desde el código nativo de cada plataforma — Fuente: https://www.jetbrains.com/help/kotlin-multiplatform-dev/multiplatform-discover-project-structure.html (OFICIAL)
- Pregunta derivada: ¿Cuál es el límite exacto de procesos de renderizado concurrentes que impone Android System WebView para orígenes distintos y qué ocurre (cola, crash o fusión) al intentar cargar diez dominios de proveedores de IA simultáneamente?
- Pregunta derivada: ¿Es viable arquitectónicamente servir las diez sesiones desde un servidor HTTP local en Rust y mostrarlas en un único webview nativo mediante iframes o un router SPA, eliminando así el cuello de botella de los límites de procesos del sistema operativo móvil?
- Pregunta derivada: ¿Cuál es la latencia real de inyección de prompts y lectura de respuestas del DOM a través del bridge de JavaScript de WKWebView y Android WebView cuando compiten diez flujos de chat concurrentes por el hilo principal?
- Pregunta derivada: Dado que el componente oficial NativeWebView de Avalonia exige licencia comercial tras la reestructuración de sus tiers, ¿qué madurez y estabilidad tiene el fork comunitario MIT para soportar los eventos táctiles y el teclado en Android e iOS?

- Pregunta derivada: ¿Cómo se compara el tamaño del binario final y el tiempo de arranque en frío de una aplicación Kotlin Multiplatform (Jetpack Compose + SwiftUI) que envuelve un núcleo Rust frente a los 11 MiB y 0,76 s medidos en Slint?

**Pregunta:** Contexto: ChatCouncil es una app de investigación que abre diez páginas web reales de proveedores de IA a la vez, cada una en su propio webview nativo con la sesión del usuario; escribe en sus campos de texto, lee sus respuestas del DOM, adjunta archivos y arma un informe. Se está reescribiendo con el núcleo en Rust y un solo código para Windows, macOS, Linux, Android e iOS (escritorio, móvil y tableta), con el rendimiento (RAM, CPU, tiempo de arranque y tamaño) como prioridad absoluta.

Ya medido en Windows, con diez webviews nativos de WebView2 en la misma máquina: Flutter con plugins que dibujan por textura gasta unos 5 puntos de CPU extra cuando las páginas redibujan; Flutter con ventanas nativas y Slint con ventanas nativas (wry) gastan casi cero en el proceso propio; Slint arranca en 0,76 s contra 3,35 s de Flutter, pesa 11 MiB contra 25,5 MiB y usa 74 MB de RAM en el proceso propio contra 122 MB. Se descartaron Dioxus (su interfaz sería un webview más) y Rune (no tiene Android ni iOS).

Pregunta: ¿qué opciones de interfaz y de manejo de varios webviews nativos simultáneos, viables hoy para esas cinco plataformas con un solo código y un núcleo en Rust, no se están considerando, y cómo se comparan en rendimiento medido con Slint y con Flutter? Para cada opción: fuentes verificables con fecha, estado real en cada plataforma (en especial varios webviews nativos simultáneos en Android e iOS), madurez, licencia y riesgos.
**Fecha:** 2026-11-10T03:36:18.180Z
**Eje de registro:** los tres (HECHOS, FUENTES, CONCLUSIONES)

## Lectura del integrador

1. TIPOS DE DIVERGENCIA
   Las divergencias fácticas, que en principio se resolverían con evidencia, incluyen las que discuten licencias o disponibilidad de Avalonia NativeWebView [H10 ✓] [H42 ✓] [H89 ✓] [H145 ✓] [H146 ✓] [H224 ✓] [H269 ✓] [H328 ✓]; las que discuten cifras o fuentes de RAM y benchmarks de egui, Slint o Tauri [H11 ✓] [H46 ✓] [H156 ✓] [H157 ✓] [H226 ✓] [H227 ✓] [H272 ✓] [H330 ✓] [H331 ✓]; las que discuten el estado de Tauri o wry en móvil, ventanas y webviews hijos [H41 ✓] [H88 ✓] [H142 ✓] [H158 ✓] [H223 ✓] [H232 ✓] [H327 ✓]; y las que discuten soporte, API o plataformas de Compose, Qt, Freya, GPUI, iced, Native Shell o ComposeNativeWebView [H43 ✓] [H44 ✓] [H45 ✓] [H90 ✓] [H91 ✓] [H144 ✓] [H147 ✓] [H149 ✓] [H153 ✓] [H154 ✓] [H228 ✓] [H229 ✓] [H230 ✓] [H231 ✓] [H270 ✓] [H271 ✓] [H274 ✓] [H329 ✓]. También son fácticas las que discrepan sobre mediciones concretas de Avalonia o sobre la verificación del código de wry [H225 ✓] [H226 ✓] [H158 ✓] [H232 ✓]. En todos estos casos hace falta un dato, una fuente primaria o una prueba ejecutable.

Las divergencias conceptuales aparecen cuando “multi-webview” se usa para cosas distintas: ventanas o escenas separadas frente a vistas hijas dentro de una misma pantalla [H87 ✓] [H88 ✓] [H142 ✓] [H143 ✓] [H327 ✓]. Ahí no basta con medir más: hay que fijar qué significa el requisito de diez webviews simultáneos y si se refiere a una cuadrícula de vistas embebidas o a ventanas del sistema.

Las divergencias de alcance agrupan casos en que unas respuestas evalúan una opción y otras la omiten o miran otra parte del problema: Native Shell [H9 ✓] [H155 ✓], Freya [H150 ✓], WaterUI [H151 ✓], Fission [H152 ✓], Dioxus [H275 ✓], y las cifras de Slint, egui o Tauri tomadas de estudios con cargas distintas [H156 ✓] [H227 ✓] [H273 ✓] [H330 ✓] [H331 ✓]. No son necesariamente contradicciones sobre el mismo hecho, sino respuestas que cubren perímetros diferentes.

Las divergencias de énfasis son las que comparten hechos pero priorizan de otro modo: probar Native Shell frente a prototipar Slint más wry [H12 ✓], migrar de Slint a egui o mantener Slint [H92 ✓], y tratar Qt como alternativa viable con costes frente a descartarlo de plano [H148 ✓]. Estas no se resuelven con un dato aislado, sino con una decisión sobre prioridades.

2. CALIDAD DE LAS CONVERGENCIAS

Las convergencias iniciales sobre wry, Tauri, iOS, Android y Makepad no traen información de fuentes en la propia tabla para H1 a H7; solo H8 registra que se citan documentación de wry, Tauri, Android Developers y fuentes sobre WKWebView/Jetsam [H1 ✓] [H2 ✓] [H3 ✓] [H4 ✓] [H5 ✓] [H6 ✓] [H7 ✓] [H8 ✓]. Por tanto, la coincidencia de H1 a H7 no puede atribuirse a fuentes independientes con lo que muestra la tabla.

En el bloque de H24 a H40, las convergencias H24 a H35 y H39 a H40 no traen información de fuentes; en cambio H36 registra Android Developers y código de Chromium, H37 registra el estudio Zenn de agosto de 2026 y H38 registra GoldStrikeArch [H24 ✓] [H25 ✓] [H26 ✓] [H27 ✓] [H28 ✓] [H29 ✓] [H30 ✓] [H31 ✓] [H32 ✓] [H33 ✓] [H34 ✓] [H35 ✓] [H36 ✓] [H37 ✓] [H38 ✓] [H39 ✓] [H40 ✓]. Esas tres sí muestran coincidencia con fuentes compartidas o identificadas, no solo coincidencia de conclusiones.

En H71 a H86, la mayoría no trae información de fuentes; H85 registra que dos respuestas respaldan la comparativa de CPU con el mismo estudio GoldStrikeArch, y H86 advierte que los benchmarks de terceros usan perímetros distintos [H71 ✓] [H72 ✓] [H73 ✓] [H74 ✓] [H75 ✓] [H76 ✓] [H77 ✓] [H78 ✓] [H79 ✓] [H80 ✓] [H81 ✓] [H82 ✓] [H83 ✓] [H84 ✓] [H85 ✓] [H86 ✓]. La convergencia H86 es especialmente cauta: no afirma que las cifras sean equivalentes, sino que no deben compararse sin ajustar el perímetro.

En H119 a H141, H126 registra el mismo benchmark Zenn, H139 registra documentación y código de wry, H140 registra documentación de Android y H141 registra documentación de Apple/WebKit y análisis de terceros; las demás convergencias de ese bloque no traen información de fuentes [H119 ✓] [H120 ✓] [H121 ✓] [H122 ✓] [H123 ✓] [H124 ✓] [H125 ✓] [H126 ✓] [H127 ✓] [H128 ✓] [H129 ✓] [H130 ✓] [H131 ✓] [H132 ✓] [H133 ✓] [H134 ✓] [H135 ✓] [H136 ✓] [H137 ✓] [H138 ✓] [H139 ✓] [H140 ✓] [H141 ✓]. Hay, por tanto, convergencias

apoyadas en fuentes compartidas y otras que solo coinciden en la conclusión.
En H203 a H222, H219 registra el benchmark Zenn, H220 registra GoldStrikeArch y H221 registra el repositorio ComposeNativeWebview; el resto no trae información de fuentes [H203 ✓] [H204 ✓] [H205 ✓] [H206 ✓] [H207 ✓] [H208 ✓] [H209 ✓] [H210 ✓] [H211 ✓] [H212 ✓] [H213 ✓] [H214 ✓] [H215 ✓] [H216 ✓] [H217 ✓] [H218 ✓] [H219 ✓] [H220 ✓] [H221 ✓] [H222 ✓]. En H256 a H268 no hay información de fuentes para ninguna de las convergencias registradas [H256 ✓] [H257 ✓] [H258 ✓] [H259 ✓] [H260 ✓] [H261 ✓] [H262 ✓] [H263 ✓] [H264 ✓] [H265 ✓] [H266 ✓] [H267 ✓] [H268 ✓].
Finalmente, en H310 a H326, H326 registra que dos respuestas citan el benchmark Zenn y el caso Sarrade; las demás convergencias de ese bloque no traen información de fuentes [H310 ✓] [H311 ✓] [H312 ✓] [H313 ✓] [H314 ✓] [H315 ✓] [H316 ✓] [H317 ✓] [H318 ✓] [H319 ✓] [H320 ✓] [H321 ✓] [H322 ✓] [H323 ✓] [H324 ✓] [H325 ✓] [H326 ✓]. Así, parte del acuerdo sobre el cuello de botella móvil y la necesidad de prototipo aparece sin fuente compartida explícita en la tabla.

3. LO QUE NADIE DIJO
   Ninguna respuesta entrega una medición propia o pública reproducible con exactamente diez webviews de proveedores reales en Android e iOS, ni con las cuatro métricas pedidas de RAM, CPU, arranque y tamaño [H21 ✓] [H22 ✓] [H62 ✓] [H63 ✓] [H64 ✓] [H113 ✓] [H198 ✓] [H199 ✓] [H252 ✓] [H254 ✓] [H351 ✓]. Tampoco hay una prueba ejecutable propia que demuestre multi-webview hijo funcional en móvil bajo wry o equivalente Rust puro [H23 ✓], ni una comparación directa de las seis opciones bajo el mismo arnés de diez webviews [H199 ✓].
   Quedan sin responder las mediciones en tableta, el coste del flujo de adjuntos de archivos, la huella total atribuible bajo carga incluyendo procesos de WebView2/WKWebView/renderizadores, y el consumo de batería o energía [H114 ✓] [H115 ✓] [H116 ✓] [H305 ✓] [H307 ✓]. Tampoco se analizan sistemáticamente los bloqueos de login en webviews embebidos ni la accesibilidad como criterio comparativo [H306 ✓] [H308 ✓]. Falta, además, una recomendación final única y común entre las respuestas [H202 ✓].
   Otras ausencias son más específicas: no hay cifras medidas de Makepad o Fission comparables con Slint y Flutter [H253 ✓], no se verificó experimentalmente que Native Shell, Avalonia NativeWebView, Uno, Qt WebView o ComposeNativeWebView mantengan diez instancias activas con sesiones y adjuntos en Android e iOS [H200 ✓], y no hay fuente primaria de Apple que cuantifique el límite exacto de procesos WebContent simultáneos [H201 ✓]. Tampoco hay mediciones en Linux con webviews embebidos ni verificación del flujo completo de adjuntar archivos y leer/escribir el DOM en las alternativas [H352 ✓] [H353 ✓].
4. QUE HARIA FALTA
   Las divergencias fácticas sobre licencias, disponibilidad y API de Avalonia, ComposeNativeWebView, Compose, Qt, Freya, GPUI, iced, Native Shell, wry y Tauri se resolverían con documentación oficial fechada, notas de release, paquetes NuGet, repositorios de código y, cuando corresponda, una prueba ejecutable [H10 ✓] [H41 ✓] [H42 ✓] [H43 ✓]

[H44 ✓] [H45 ✓] [H89 ✓] [H90 ✓] [H91 ✓] [H144 ✓] [H145 ✓] [H146 ✓] [H147 ✓] [H149 ✓] [H153 ✓] [H154 ✓] [H157 ✓] [H158 ✓] [H223 ✓] [H224 ✓] [H225 ✓] [H226 ✓] [H228 ✓] [H229 ✓] [H230 ✓] [H231 ✓] [H232 ✓] [H269 ✓] [H270 ✓] [H271 ✓] [H274 ✓] [H328 ✓] [H329 ✓].
Las divergencias sobre cifras de RAM, fuentes de benchmarks y versiones de wry necesitan el mismo arnés, la misma carga y la misma plataforma, o al menos la separación explícita de perímetros [H11 ✓] [H46 ✓] [H156 ✓] [H157 ✓] [H226 ✓] [H227 ✓] [H272 ✓] [H330 ✓] [H331 ✓].
Las divergencias conceptuales sobre Tauri y wry no se resuelven solo acumulando fuentes: requieren una definición operativa de “multi-webview” y una decisión sobre si el requisito exige vistas hijas en una pantalla o acepta ventanas/escenas separadas [H87 ✓] [H88 ✓] [H142 ✓] [H143 ✓] [H327 ✓]. Aun así, una lectura del código de wry y una prueba en dispositivo ayudarían a separar la discusión terminológica de la capacidad real [H158 ✓] [H232 ✓].
Las divergencias de alcance sobre Native Shell, Freya, WaterUI, Fission, Dioxus y las cifras de Slint, egui o Tauri necesitan que las respuestas declaren qué evaluaron y bajo qué carga, o que se haga una evaluación directa con el mismo perímetro [H9 ✓] [H150 ✓] [H151 ✓] [H152 ✓] [H155 ✓] [H156 ✓] [H227 ✓] [H273 ✓] [H275 ✓] [H330 ✓] [H331 ✓]. Sin ese ajuste, no son contradicciones resolubles con un dato único.
Las divergencias de énfasis sobre Native Shell, egui frente a Slint y Qt no se resuelven con más información: exigen una decisión de quien lee sobre prioridades, tolerancia a licencias, coste de compilación y apetito de prototipo [H12 ✓] [H92 ✓] [H148 ✓]. La tabla no permite decidir por sí sola cuál de esas prioridades debe pesar más.

5. QUE CONVIENE RESCATAR
   Lo firme es que wry y Tauri no ofrecen, según la mayoría, webviews hijos dentro de una misma pantalla en Android e iOS; lo que existe en móvil se describe como ventanas o escenas separadas, no como una cuadrícula de diez vistas embebidas [H1 ✓] [H71 ✓] [H203 ✓] [H256 ✓] [H310 ✓] [H311 ✓]. También es firme que el cuello de botella decisivo está en el modelo de procesos y memoria del sistema móvil, no solo en el toolkit de UI [H4 ✓] [H267 ✓] [H323 ✓]. Y ninguna respuesta encontró un benchmark público que reproduzca el caso exacto de diez proveedores reales en móvil, por lo que las mediciones de Windows del usuario siguen siendo el único dato directo de esa clase [H21 ✓] [H62 ✓] [H76 ✓] [H257 ✓] [H325 ✓].
   Las contradicciones que más importan son, primero, la licencia y disponibilidad de Avalonia NativeWebView, porque de ello depende si es una opción gratuita o comercial y si sirve como plan B real [H42 ✓] [H89 ✓] [H145 ✓] [H224 ✓] [H269 ✓] [H328 ✓]. Segundo, el significado de “multi-webview” en Tauri, porque tomar la versión de “ventanas móviles” como si fueran vistas hijas puede llevar a una arquitectura que no cumple el requisito de diez en una pantalla [H87 ✓] [H88 ✓] [H142 ✓] [H143 ✓] [H223 ✓] [H327 ✓]. Tercero, el soporte oficial de egui en iOS, porque afecta si se puede considerar una alternativa de cinco plataformas o solo de escritorio [H90 ✓] [H154 ✓] [H229 ✓]. Cuarto, la madurez y licencia de ComposeNativeWebView, porque unas respuestas lo presentan como vía empaquetada y otras como candidato joven sin verificar

[H228 ✓] [H274 ✓] [H329 ✓]. Quinto, las cifras de RAM de egui, Slint y Tauri, porque mezclan cargas y perímetros y pueden inducir a una comparación falsa [H11 ✓] [H227 ✓] [H272 ✓] [H330 ✓] [H331 ✓].
Lo que sigue abierto es lo que más falta para decidir: no hay prueba en dispositivos físicos Android e iOS con diez páginas reales autenticadas, sesiones, DOM y adjuntos [H63 ✓] [H198 ✓] [H254 ✓] [H304 ✓]; no hay mediciones en tableta [H114 ✓]; no hay coste medido del flujo de adjuntos ni de lectura/escritura del DOM [H115 ✓] [H305 ✓] [H353 ✓]; y no hay una recomendación final común entre las respuestas [H202 ✓].
Conviene verificar primero, en fuentes primarias fechadas, las licencias y el estado real de Avalonia NativeWebView, ComposeNativeWebView, Tauri/wry y egui en iOS; después, con código y prueba ejecutable, si wry permite o no vistas hijas en móvil y qué significa exactamente el soporte multi-ventana de Tauri [H158 ✓] [H232 ✓] [H225 ✓] [H228 ✓] [H90 ✓]. El paso siguiente que la tabla respalda es un prototipo propio en dispositivos Android e iOS antes de reescribir la interfaz, porque las respuestas coinciden en que no existe la medición pública que haría innecesario ese prototipo [H78 ✓] [H268 ✓] [H323 ✓] [H325 ✓].

## Participación de operadores

**ChatGPT** — ok: 47 hallazgos
**Gemini** — ok: 53 hallazgos
**Claude** — ok: 54 hallazgos
**Grok** — ok: 23 hallazgos
**Mistral** — ok: 48 hallazgos
**Kimi** — ok: 45 hallazgos
**Perplexity** — ok: 84 hallazgos

## Hallazgos referenciados

**H10** — DIVERGENCIA · HECHOS
Registrado por: Grok
Respuestas que lo sostienen: Perplexity, Kimi, Mistral, Claude, Gemini

> discrepan en el estado exacto de Avalonia NativeWebView: unas lo marcan disponible en las cinco con licencia MIT/comercial mixta y composición nativa; otras señalan offscreen en Linux o licencia Accelerate de pago

**H42** — DIVERGENCIA · HECHOS
Registrado por: ChatGPT
Respuestas que lo sostienen: Gemini, Kimi

> P2 incluye Avalonia 12 entre las versiones cuyo NativeWebView requiere la licencia comercial Accelerate. P6 afirma que el control pasó a ser de código abierto desde Avalonia 12 y presenta la opción como MIT.

**H89** — DIVERGENCIA · HECHOS
Registrado por: Mistral
Respuestas que lo sostienen: ChatGPT, Gemini, Claude, Kimi

> P1 y P6 afirman que el NativeWebView de Avalonia 12 es de código abierto bajo MIT; P2 y P3 sostienen que forma parte del paquete comercial Avalonia Accelerate (P3 cita una alternativa comunitaria MIT en 0.1.0-alpha como única vía gratuita)

**H145** — DIVERGENCIA · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT, Claude

> P1 reporta que Avalonia 12 NativeWebView es MIT, incluido el componente web; P3 afirma que el paquete oficial requiere la licencia de pago Avalonia Accelerate y que solo existe una alternativa comunitaria MIT. operacion-P7-2026-11-09-2302.txt

**H146** — DIVERGENCIA · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT, Kimi

> P1 atribuye a Avalonia 12 una licencia MIT para el componente web; P6 también la llama MIT, pero afirma que NativeWebView forma parte del paquete comercial Avalonia Accelerate. operacion-P7-2026-11-09-2302.txt

**H224** — DIVERGENCIA · HECHOS
Registrado por: Gemini
Respuestas que lo sostienen: ChatGPT, Claude, Kimi

> P1 y P6 afirman que NativeWebView de Avalonia 12 (abril de 2026) es open source bajo licencia MIT, mientras que P3 afirma que el control oficial requiere la licencia comercial de pago Avalonia Accelerate y que la alternativa MIT es un paquete comunitario en versión 0.1.0-alpha.1.

**H269** — DIVERGENCIA · HECHOS
Registrado por: Claude
Respuestas que lo sostienen: ChatGPT, Kimi, Gemini

> la licencia del componente NativeWebView de Avalonia: P1 y P6 dicen MIT, P2 dice que forma parte del paquete comercial Avalonia Accelerate y cita la FAQ oficial (6 oct 2026)

**H328** — DIVERGENCIA · HECHOS
Registrado por: Kimi
Respuestas que lo sostienen: ChatGPT, Claude

> licencia del NativeWebView de Avalonia: P1 lo da por MIT incluido el componente web en Avalonia 12; P3 afirma que el paquete oficial exige la licencia comercial Accelerate y que la vía MIT es una alternativa comunitaria en alpha

**H11** — DIVERGENCIA · HECHOS
Registrado por: Grok
Respuestas que lo sostienen: Kimi, Mistral, Gemini

> difieren en cifras de RAM de egui vs Slint (empate aproximado en un estudio vs 135 MB → 30 MB en otro caso documentado)

**H46** — DIVERGENCIA · FUENTES
Registrado por: ChatGPT
Respuestas que lo sostienen: Gemini, Claude, Mistral, Kimi

> Para el modelo de procesos y el costo de WKWebView, P2 recurre a Embrace y SegmentFault, P3 a Apple y Embrace, P5 a BestHub y Cross Mini App, y P6 a Embrace y un issue de Capacitor. Las cifras por instancia proceden de referencias distintas.

**H156** — DIVERGENCIA · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT, Kimi

> P1 cita guibench con Slint en 51,3 ms de arranque y 47,4 MiB de RAM; P6 cita un estudio distinto de 15 frameworks con Slint en 102,9–124,4 MB según escenario. Ambas son mediciones públicas, pero usan metodologías y cargas diferentes. operacion-P7-2026-11-09-2302.txt

**H157** — DIVERGENCIA · FUENTES
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT, Mistral, Kimi

> P1 usa guibench, el informe del fabricante de Native Shell y la medición de ComposeNativeWebView; P5 y P6 usan principalmente el benchmark Zenn de agosto de 2026 y el caso egui→Slint de septiembre de 2026. operacion-P7-2026-11-09-2302.txt

**H226** — DIVERGENCIA · HECHOS
Registrado por: Gemini
Respuestas que lo sostienen: ChatGPT, Claude, Kimi

> P3 afirma que no encontró mediciones de rendimiento para Avalonia, mientras que P1 y P6 sí reportan mediciones de RAM de terceros (agosto de 2026) y P6 añade las métricas oficiales con NativeAOT en Android (460 ms de arranque y <0,01% CPU idle).

**H227** — DIVERGENCIA · HECHOS
Registrado por: Gemini
Respuestas que lo sostienen: ChatGPT, Mistral, Kimi

> Aunque las tres extraen datos del mismo benchmark de macOS del 15 de agosto de 2026, reportan cifras distintas: para Slint en 4 paneles, P1 da 133,0 MB y P6 da 116,5 MB (y P5 da 106–124 MB pico general); para Flutter, P1 da 363,8 MB, P6 da 165,0–246,7 MB y P5 da 176–199 MB; para Tauri, P5 da 309–348 MB pico y P6 da 332,7–739,0 MB.

**H272** — DIVERGENCIA · HECHOS
Registrado por: Claude
Respuestas que lo sostienen: Grok, Kimi

> cifras de RAM para egui en benchmarks de terceros: P4 dice “a menudo mejor o comparable a Slint en idle/simple”; P6 cita el benchmark Zenn (95-157 MB) y el caso Sarrade (135 MB egui vs 30 MB Slint con software renderer), concluyendo que Slint es claramente mejor

**H330** — DIVERGENCIA · FUENTES
Registrado por: Kimi
Respuestas que lo sostienen: ChatGPT, Gemini, Mistral

> RAM de egui frente a Slint: P1 (guibench, jul 2026) las da casi iguales (48,4 vs 47,4 MiB); P2 reporta 45-65 MB para egui; P5 (Zenn, ago 2026) da egui 148-157 MB vs Slint 106-124 MB;

> perímetros distintos con conclusiones incompatibles

**H331** — DIVERGENCIA · FUENTES
Registrado por: Kimi
Respuestas que lo sostienen: ChatGPT, Grok, Mistral

> RAM de Tauri: P1 da 59,0 MiB (UI sola, guibench); P4 da 30-50+ MB en idle; P5 da 309-348 MB de pico con webviews reales (Zenn); cifras correctas según su perímetro pero no comparables entre sí

**H41** — DIVERGENCIA · HECHOS
Registrado por: ChatGPT
Respuestas que lo sostienen: Claude, Grok, Mistral, Kimi, Perplexity

> P6 afirma que Tauri carece de multiventana en Android. Las otras cuatro describen su incorporación en 2026. La discrepancia se refiere a ventanas separadas de Android.

**H88** — DIVERGENCIA · HECHOS
Registrado por: Mistral
Respuestas que lo sostienen: ChatGPT, Gemini, Claude, Grok

> P2 sostiene que el multiwebview de Tauri no está disponible en móvil citando el issue #11528 (octubre de 2024), mientras P1, P3 y P4 documentan multi-ventana móvil fusionada en marzo de 2026 (Tauri 2.11); la discrepancia puede deberse a que P2 describe webviews hijos en una ventana y los demás el multi-ventana, pero como quedan escritas las afirmaciones son incompatibles

**H142** — DIVERGENCIA · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT, Grok

> P1 afirma que wry no soporta build_as_child en Android ni iOS; P4 afirma que Tauri 2 logró multi-webview/multi-window móvil en marzo de 2026 mediante Activity Embedding y UIScene. La diferencia es real pero depende de si “multi-webview” significa vistas hijas en una pantalla o ventanas/escenas separadas. operacion-P7-2026-11-09-2302.txt

**H158** — DIVERGENCIA · FUENTES
Registrado por: Perplexity
Respuestas que lo sostienen: Claude, Mistral

> P3 revisa el código fuente de wry y concluye con confianza alta que no soporta varios webviews móviles; P5 se basa en la documentación de wry/Tauri y declara que no verificó con una prueba ejecutable si wry permite varias vistas en móvil. operacion-P7-2026-11-09-2302.txt

**H223** — DIVERGENCIA · HECHOS
Registrado por: Gemini
Respuestas que lo sostienen: ChatGPT, Claude, Grok, Mistral, Kimi, Perplexity

> Mientras P1, P3, P4, P5 y P7 afirman que Tauri 2 añadió soporte multi-ventana en Android mediante Activity Embedding en marzo/abril de 2026 (PR #14484 / v2.11), P6 afirma que en Android no existe soporte multi-ventana en Tauri 2.

**H232** — DIVERGENCIA · HECHOS
Registrado por: Gemini
Respuestas que lo sostienen: ChatGPT, Claude, Mistral, Kimi, Perplexity

> P1, P3 y P6 dan por verificado en la documentación y código fuente de wry 0.57.1 que wry no soporta múltiples webviews hijos en una pantalla móvil, mientras que P5 y P7 declaran como incógnita no verificada si Slint + wry permite incrustar múltiples webviews en móvil y proponen hacer una prueba ejecutable para averiguarlo.

**H327** — DIVERGENCIA · HECHOS
Registrado por: Kimi
Respuestas que lo sostienen: Claude, Grok, Perplexity

> estado del multi-webview móvil en wry/Tauri: P4 afirma soporte side-by-side en tablets/iPad desde ~mar 2026 (PRs #1633/#14484); P7 dice que esas APIs son para ventanas separadas, no vistas hijas; P3, tras leer el código fuente de wry 0.57.1, afirma un solo webview por Activity y set_bounds sin efecto en Android

**H43** — DIVERGENCIA · HECHOS
Registrado por: ChatGPT
Respuestas que lo sostienen: Claude, Kimi

> P3 sitúa la estabilización de Compose Multiplatform para iOS en la versión 1.8.0 de mayo de 2025. P6 la sitúa en 2024.

**H44** — DIVERGENCIA · HECHOS
Registrado por: ChatGPT
Respuestas que lo sostienen: Mistral, Perplexity

> P5 dice que el esfuerzo comunitario iced-mobile solo cubre Android. P7 describe iced_mobile como una prueba de concepto para Android e iOS.

**H45** — DIVERGENCIA · HECHOS
Registrado por: ChatGPT
Respuestas que lo sostienen: Grok, Perplexity

> P4 describe incrustación de webviews nativos en Freya para escritorio mediante freya-webview y wry. P7 marca Freya con «No» en sus columnas sobre webviews.

**H90** — DIVERGENCIA · HECHOS
Registrado por: Mistral
Respuestas que lo sostienen: ChatGPT, Gemini, Grok, Kimi, Perplexity

> P4 presenta iOS de egui como "soporte reportado" y P2 dice que eframe/iced "arrancan en Android e iOS"; P1, P6 y P7 documentan que iOS no es target oficial (issue #3117 abierto; emilk en marzo de 2026 pregunta qué PR se habría fusionado) y exige integración propia

**H91** — DIVERGENCIA · HECHOS
Registrado por: Mistral
Respuestas que lo sostienen: Grok, Perplexity

> P4 sostiene que Freya ofrece multi-webview nativo de escritorio vía freya-webview (wry); la tabla de P7 marca a Freya sin soporte de varios webviews nativos

**H144** — DIVERGENCIA · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT, Claude

> P1 considera que Native Shell puede alojar WebView y WKWebView dentro de la jerarquía nativa en Android e iOS; P3 no menciona Native Shell y sostiene que la capa móvil debe escribirse a mano en Kotlin/Swift. operacion-P7-2026-11-09-2302.txt

**H147** — DIVERGENCIA · HECHOS
Registrado por: Perplexity

Respuestas que lo sostienen: ChatGPT, Claude

> P1 describe Avalonia NativeWebView como un control embebido con API común de JavaScript y mensajes; P3 señala que en Android/iOS las vistas nativas siempre quedan encima del contenido de Avalonia y no permiten superponer controles de Avalonia. operacion-P7-2026-11-09-2302.txt

**H149** — DIVERGENCIA · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT, Claude

> P1 dice que Qt Bridge for Rust salió como versión preliminar el 27 de octubre de 2026 sin lista de plataformas ni mención de Android/iOS; P3 añade que ese puente solo soporta Windows, macOS y Linux. operacion-P7-2026-11-09-2302.txt

**H153** — DIVERGENCIA · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: Grok, Kimi

> P4 afirma que GPUI tiene composición nativa de webviews emergente en escritorio; P6 afirma que gpui es solo escritorio y que gpui-webview hereda las limitaciones de wry, sin móvil. operacion-P7-2026-11-09-2302.txt

**H154** — DIVERGENCIA · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: Grok, Mistral, Kimi

> P4 considera que egui tiene soporte móvil “menos maduro” pero presente; P5 y P6 concluyen que egui no tiene soporte iOS oficial y que su integración móvil es incompleta. operacion-P7-2026-11-09-2302.txt

**H228** — DIVERGENCIA · HECHOS
Registrado por: Gemini
Respuestas que lo sostienen: ChatGPT, Claude, Mistral

> P3 afirma que los webviews nativos de escritorio no están disponibles en Compose Multiplatform, mientras que P1 y P5 documentan la existencia de ComposeNativeWebview para escritorio y móvil; a su vez, P5 afirma que ComposeNativeWebview se basa en wry y que

> no pudo verificar su licencia ni su soporte iOS, mientras que P1 verifica que es Apache-2.0 (v1.0.3, 3 sep 2026), que usa Nucleus Tao en escritorio y que sí soporta iOS (WKWebView).

**H229** — DIVERGENCIA · HECHOS
Registrado por: Gemini
Respuestas que lo sostienen: ChatGPT, Grok, Mistral, Kimi, Perplexity

> P4 y P5 afirman que egui / eframe tiene soporte para iOS (P5 cita eframe 0.31.0), mientras que P1, P6 y P7 afirman que eframe no tiene soporte oficial para iOS (P6 cita el issue #3117 abierto en marzo de 2026) y que requiere integración manual con winit.

**H230** — DIVERGENCIA · HECHOS
Registrado por: Gemini
Respuestas que lo sostienen: Grok, Perplexity

> Respecto a Freya, P7 afirma en su tabla que no tiene soporte de múltiples webviews nativos, mientras que P4 documenta el crate freya-webview (basado en wry) que soporta múltiples webviews nativos como elementos de layout en escritorio (Windows, macOS y Linux X11).

**H231** — DIVERGENCIA · HECHOS
Registrado por: Gemini
Respuestas que lo sostienen: Grok, Mistral, Kimi, Perplexity

> Respecto a GPUI, P5 y P7 afirman que no tiene integración de webviews, mientras que P4 y P6 documentan la existencia de soporte experimental de webviews en escritorio mediante gpui-webview / gpui-wry.

**H270** — DIVERGENCIA · HECHOS
Registrado por: Claude
Respuestas que lo sostienen: ChatGPT, Gemini

> el comportamiento de Avalonia NativeWebView en Linux: P1 dice que la ruta WPE compone fuera de pantalla por memoria compartida (DMA-BUF) sin lectura de vuelta a CPU; P2 dice que renderiza offscreen y reintroduce el costo de CPU por copia de texturas, citando la misma documentación oficial

**H271** — DIVERGENCIA · HECHOS
Registrado por: Claude
Respuestas que lo sostienen: Grok, Gemini

> la versión de wry utilizada como referencia: P2 analiza wry 0.54.0 (enero 2026) y P1 menciona wry 0.57.1 (octubre 2026); P4 no precisa versión pero describe funcionalidades de 2026

**H274** — DIVERGENCIA · HECHOS
Registrado por: Claude
Respuestas que lo sostienen: Mistral, Gemini

> la cobertura de ComposeNativeWebView: P2 dice que es “la única combinación que hoy ofrece múltiples webviews nativos sin copia de texturas empaquetados de serie en las 5 plataformas”; P5 dice que es “proyecto joven (182 estrellas)” con licencia no verificada y costo heredado de la JVM

**H329** — DIVERGENCIA · FUENTES
Registrado por: Kimi
Respuestas que lo sostienen: ChatGPT, Gemini, Claude

> licencia de ComposeNativeWebView: P1 y P2 afirman Apache-2.0; P3 declara no haberla verificado

**H225** — DIVERGENCIA · FUENTES
Registrado por: Gemini
Respuestas que lo sostienen: ChatGPT, Claude, Kimi

> Para sustentar el estado y licencia de NativeWebView en Avalonia, P1 y P6 citan el anuncio de Avalonia 12 (avaloniaui.net/blog/avalonia-12), mientras que P3 cita la documentación comercial de Avalonia Accelerate y el paquete NuGet Avalonia.Controls.WebView 11.3.15.

**H87** — DIVERGENCIA · CONCLUSIONES
Registrado por: Mistral
Respuestas que lo sostienen: Claude, Grok, Kimi, Perplexity

> P4 afirma que Tauri 2 + wry es la opción más madura y viable para multi-webview nativo simultáneo en las cinco plataformas; P3, P6 y P7 concluyen lo contrario: Tauri hereda las limitaciones de wry (sin webviews hijos en móvil, API inestable incluso en escritorio) y no resuelve el caso

**H143** — DIVERGENCIA · HECHOS
Registrado por: Perplexity

Respuestas que lo sostienen: Grok, Mistral, Kimi

> P4 presenta Tauri 2 como la opción más madura y viable para multi-webview nativos simultáneos en las cinco plataformas; P5 y P6 concluyen que Tauri no resuelve diez webviews simultáneos en una sola pantalla de teléfono. operacion-P7-2026-11-09-2302.txt

**H9** — DIVERGENCIA · HECHOS
Registrado por: Grok
Respuestas que lo sostienen: ChatGPT, Kimi, Mistral, Gemini

> P1 presenta Native Shell (nsh 0.4.0, nov 2026) como la alternativa nueva prioritaria con backends nativos en las cinco plataformas; las demás no la mencionan o la omiten

**H155** — DIVERGENCIA · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT, Mistral

> P1 reporta cifras de Native Shell en Windows de aproximadamente 14 MB de memoria privada, 60 MB de working set y 15 MB de ejecutable; P5 no menciona Native Shell en absoluto. operacion-P7-2026-11-09-2302.txt

**H150** — DIVERGENCIA · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: Grok, Mistral

> P4 estima que Freya tiene rendimiento esperado cercano a Slint y superior a Flutter textura; P5 no incluye a Freya entre las opciones viables y solo analiza Dioxus Native/Blitz, Makepad, GPUI e iced como descalificados. operacion-P7-2026-11-09-2302.txt

**H151** — DIVERGENCIA · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT, Grok

> P1 reporta que WaterUI tiene un WebView con JavaScript y menciona motores CEF o WPE; P4 no evalúa WaterUI y lo agrupa entre opciones incompletas o demasiado inmaduras. operacion-P7-2026-11-09-2302.txt

**H152** — DIVERGENCIA · HECHOS
Registrado por: Perplexity

Respuestas que lo sostienen: Grok, Mistral

> P4 afirma que Fission reclama las cinco plataformas y evalúa su rendimiento como desconocido; P5 no considera Fission en su panorama y se concentra en Compose, Qt, Avalonia, egui y caparazón nativo. operacion-P7-2026-11-09-2302.txt

**H275** — DIVERGENCIA · HECHOS
Registrado por: Claude
Respuestas que lo sostienen: Perplexity, Gemini

> evaluación de Dioxus: P7 no reevalúa el descarte de Dioxus; P2 argumenta que el descarte se basó en una premisa que ya no aplica porque Dioxus Native (Blitz) renderiza sin webview, aunque reconoce que Blitz no tiene motor JavaScript

**H273** — DIVERGENCIA · HECHOS
Registrado por: Claude
Respuestas que lo sostienen: Gemini, Kimi

> cifras de RAM reportadas para egui: P2 dice 45-65 MB citando Reddit (2025); P6 dice 94-157 MB citando el benchmark Zenn (agosto 2026, macOS, app multi-módulo); ambos números provienen de mediciones distintas con cargas diferentes, pero P2 los presenta como rango general sin matizar la diferencia de carga

**H12** — DIVERGENCIA · CONCLUSIONES
Registrado por: Grok
Respuestas que lo sostienen: ChatGPT, Perplexity, Kimi

> P1 prioriza probar Native Shell en paralelo con host mínimo; P7/P6 priorizan prototipo de Slint + wry o contenedor nativo propio y sitúan Native Shell/Makepad más abajo o como apuesta de riesgo

**H92** — DIVERGENCIA · CONCLUSIONES
Registrado por: Mistral
Respuestas que lo sostienen: Gemini, Kimi

> P2 concluye que conviene evaluar cambiar Slint por egui (licencia sin atribución; compilación de 2-15 s frente a 42-271 s de Slint); P6 concluye que egui/iced no aportan nada que justifique migrar y cita un benchmark donde el modelo retenido de Slint escala mejor (4 µs frente a 32 ms por edición)

**H148** — DIVERGENCIA · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: Claude, Kimi

> P3 dice que Qt WebView usa el webview del sistema en cuatro plataformas, con Qt WebEngine solo en Linux; P6 afirma que Qt WebEngine no existe para Android ni iOS por restricción de plataforma y que debe usarse Qt WebView. Ambos coinciden en lo esencial, pero P6 lo formula como descarte más categórico. operacion-P7-2026-11-09-2302.txt

**H1** — CONVERGENCIA · HECHOS
Registrado por: Grok
Respuestas que lo sostienen: Perplexity, ChatGPT, Kimi, Mistral, Claude, Gemini

> todas afirman que wry/Tauri no soportan múltiples webviews hijos en la misma ventana/actividad en Android e iOS (build_as_child o equivalente no soportado o limitado a escritorio)

**H2** — CONVERGENCIA · HECHOS
Registrado por: Grok
Respuestas que lo sostienen: Perplexity, Kimi, Mistral, Claude, Gemini

> todas reportan que en iOS cada WKWebView genera procesos WebContent propios y que mantener muchas instancias calientes es inviable por Jetsam/memoria (costos reportados 50-200+ MB por vista)

**H3** — CONVERGENCIA · HECHOS
Registrado por: Grok
Respuestas que lo sostienen: Perplexity, Kimi, Mistral, Claude, Gemini

> todas reportan que en Android las WebView de una app comparten un único proceso renderizador y que se requiere destrucción explícita

**H4** — CONVERGENCIA · CONCLUSIONES
Registrado por: Grok
Respuestas que lo sostienen: Perplexity, Kimi, Mistral, Claude, Gemini

> todas concluyen que el cuello de botella decisivo para diez webviews simultáneos está en el modelo de procesos/composición móvil del SO, no en el toolkit de UI

**H5** — CONVERGENCIA · CONCLUSIONES
Registrado por: Grok
Respuestas que lo sostienen: Perplexity, Kimi, Mistral, Gemini

> recomiendan mantener Slint (o equivalente Rust ligero) en escritorio y construir o usar un contenedor nativo (Kotlin/Swift) o pool limitado (2-4 vistas) para móvil

**H6** — CONVERGENCIA · HECHOS
Registrado por: Grok
Respuestas que lo sostienen: Perplexity, Kimi, Mistral, Gemini

> todas tratan Makepad como opción Rust de cinco plataformas pero sin integración lista de webview nativo

**H7** — CONVERGENCIA · HECHOS
Registrado por: Grok
Respuestas que lo sostienen: Perplexity, Kimi, Mistral, Claude, Gemini

> todas tratan Tauri/wry como útil para orquestación de webviews en escritorio pero insuficiente o inestable para multi-webview hijo en móvil

**H8** — CONVERGENCIA · FUENTES
Registrado por: Grok
Respuestas que lo sostienen: Perplexity, Kimi, Mistral, Claude, Gemini

> citan documentación de wry (docs.rs / README), Tauri (v2 multi-window / unstable), Android Developers (webview-memory) y fuentes sobre WKWebView/Jetsam

**H24** — CONVERGENCIA · HECHOS
Registrado por: ChatGPT
Respuestas que lo sostienen: Gemini, Claude, Grok, Mistral, Kimi, Perplexity

> Las seis usan como referencia las mediciones de Windows aportadas en la pregunta. Ninguna reproduce esa carga de diez WebView2 con una alternativa.

**H25** — CONVERGENCIA · HECHOS
Registrado por: ChatGPT
Respuestas que lo sostienen: Gemini, Mistral, Kimi, Perplexity

> Los cuatro sitúan los webviews hijos de Tauri dentro de una ventana de escritorio detrás de una API inestable.

**H26** — CONVERGENCIA · HECHOS
Registrado por: ChatGPT
Respuestas que lo sostienen: Gemini, Claude, Mistral, Kimi

> Los cuatro coinciden en que wry no ofrece en móvil una API funcional para colocar varios webviews hijos con geometría independiente dentro de una misma pantalla.

**H27** — CONVERGENCIA · HECHOS
Registrado por: ChatGPT
Respuestas que lo sostienen: Claude, Grok, Mistral, Perplexity

> Los cuatro describen soporte multiventana móvil en Tauri durante 2026. P4, P5 y P7 identifican Activity Embedding en Android y escenas en el ecosistema Apple.

**H28** — CONVERGENCIA · HECHOS
Registrado por: ChatGPT
Respuestas que lo sostienen: Gemini, Mistral, Kimi

> Los tres distinguen el soporte de webviews hijos de wry en Linux/X11 de la limitación en Wayland. P2 y P6 especifican la vía alternativa con un contenedor GTK.

**H29** — CONVERGENCIA · HECHOS
Registrado por: ChatGPT
Respuestas que lo sostienen: Gemini, Claude, Mistral, Kimi

> Los cuatro describen un renderizador habitualmente compartido por los WebView de una app Android y procesos WebContent separados en iOS. P3 añade excepciones al modelo de Android.

**H30** — CONVERGENCIA · HECHOS
Registrado por: ChatGPT
Respuestas que lo sostienen: Gemini, Mistral, Kimi, Perplexity

> Los cuatro identifican Makepad como una UI con cobertura de las cinco plataformas que requiere desarrollar la integración de los webviews nativos.

**H31** — CONVERGENCIA · HECHOS
Registrado por: ChatGPT
Respuestas que lo sostienen: Gemini, Grok, Mistral, Kimi, Perplexity

> Los cinco coinciden en que egui/eframe no proporciona la incrustación de webviews nativos que necesita ChatCouncil.

**H32** — CONVERGENCIA · HECHOS
Registrado por: ChatGPT
Respuestas que lo sostienen: Grok, Mistral, Kimi, Perplexity

> Los cuatro describen la vía móvil de iced como experimental, comunitaria o sin soporte oficial suficiente para el producto.

**H33** — CONVERGENCIA · HECHOS
Registrado por: ChatGPT
Respuestas que lo sostienen: Gemini, Grok, Mistral, Kimi, Perplexity

> Los cinco consideran insuficiente el estado móvil de GPUI para usarlo como base de producción en las cinco plataformas.

**H34** — CONVERGENCIA · HECHOS
Registrado por: ChatGPT
Respuestas que lo sostienen: Gemini, Mistral

> Ambas identifican que Blitz permite dibujar la UI de Dioxus sin sumar un webview y que su falta de motor JavaScript impide usarlo para cargar las páginas reales de los proveedores.

**H35** — CONVERGENCIA · HECHOS
Registrado por: ChatGPT
Respuestas que lo sostienen: Gemini, Claude, Mistral

> Las tres indican que Qt WebView depende de Qt WebEngine, con Chromium, en Linux. P5 distingue además WebView2 y WebEngine como opciones en Windows.

**H36** — CONVERGENCIA · FUENTES
Registrado por: ChatGPT
Respuestas que lo sostienen: Gemini, Claude, Mistral, Kimi

> Las cuatro recurren a Android Developers para el modelo de memoria de WebView. P2 y P3 también remiten al código de Chromium.

**H37** — CONVERGENCIA · FUENTES
Registrado por: ChatGPT
Respuestas que lo sostienen: Mistral, Kimi

> Ambas usan el estudio de Zenn de agosto de 2026 para comparar RAM de varias interfaces, con escenarios distintos del banco de diez webviews de ChatCouncil.

**H38** — CONVERGENCIA · FUENTES
Registrado por: ChatGPT
Respuestas que lo sostienen: Grok, Perplexity

> Ambas recurren al informe de GoldStrikeArch. P7 explicita sus cifras promedio de CPU para Tauri, egui y Slint.

**H39** — CONVERGENCIA · CONCLUSIONES
Registrado por: ChatGPT
Respuestas que lo sostienen: Gemini, Claude, Mistral, Kimi, Perplexity

> Las cinco separan la elección de la UI del problema de alojar webviews en móvil. Todas contemplan integración nativa por plataforma como vía principal o de contingencia.

**H40** — CONVERGENCIA · CONCLUSIONES
Registrado por: ChatGPT
Respuestas que lo sostienen: Gemini, Claude, Mistral, Kimi

> Las cuatro prevén presión de memoria al mantener diez páginas pesadas activas en un iPhone. P2 y P6 proponen reutilizar instancias, P3 propone medir la supervivencia y P5 plantea un ensayo escalonado.

**H71** — CONVERGENCIA · HECHOS
Registrado por: Mistral
Respuestas que lo sostienen: ChatGPT, Gemini, Claude, Kimi, Perplexity

> coinciden en que wry/Tauri no admite webviews hijos dentro de una misma ventana en Android ni iOS (wry lo documenta como "Unsupported"; en Android hay un solo webview por

> Activity) y que "ventanas nativas" de escritorio no equivale a vistas nativas embebidas en la jerarquía de vistas de móvil

**H72** — CONVERGENCIA · HECHOS
Registrado por: Mistral
Respuestas que lo sostienen: ChatGPT, Claude, Grok, Perplexity

> coinciden en que Tauri añadió multi-ventana móvil en marzo de 2026 (PR #14484, línea 2.11; Activity Embedding en Android y UIScene en iOS), útil para ventanas separadas en tabletas pero no para diez webviews en una misma pantalla de teléfono

**H73** — CONVERGENCIA · HECHOS
Registrado por: Mistral
Respuestas que lo sostienen: Gemini, Kimi

> coinciden en que build_as_child de wry en Linux solo funciona bajo X11 y que Wayland exige un contenedor GTK (build_gtk sobre gtk::Fixed) o forzar XWayland

**H74** — CONVERGENCIA · HECHOS
Registrado por: Mistral
Respuestas que lo sostienen: ChatGPT, Gemini, Claude, Kimi

> coinciden en que Android otorga a la app un único proceso renderizador compartido por todos sus WebView (guía oficial de agosto de 2026), con muerte colectiva de las vistas si cae y manejo obligatorio de onRenderProcessGone

**H75** — CONVERGENCIA · HECHOS
Registrado por: Mistral
Respuestas que lo sostienen: ChatGPT, Gemini, Claude, Kimi

> coinciden en el modelo de procesos de iOS: cada WKWebView tiene procesos propios, el sistema puede matar los de vistas no visibles dejándolas en blanco, y se recomienda compartir un WKProcessPool; citan entre 120 y 250 MB por vista

**H76** — CONVERGENCIA · HECHOS
Registrado por: Mistral
Respuestas que lo sostienen: ChatGPT, Gemini, Claude, Grok, Kimi, Perplexity

> las seis afirman que no existe ninguna medición pública de diez webviews simultáneos comparable con las mediciones de Windows del usuario

**H77** — CONVERGENCIA · CONCLUSIONES
Registrado por: Mistral
Respuestas que lo sostienen: ChatGPT, Gemini, Claude, Grok, Kimi, Perplexity

> las seis coinciden en conservar Slint como base de referencia: ninguna alternativa iguala el perfil medido de Slint + wry en el proceso propio en escritorio

**H78** — CONVERGENCIA · CONCLUSIONES
Registrado por: Mistral
Respuestas que lo sostienen: ChatGPT, Gemini, Claude, Grok, Kimi, Perplexity

> las seis concluyen que la decisión arquitectónica exige un prototipo propio medido en dispositivos físicos antes de reescribir la interfaz

**H79** — CONVERGENCIA · HECHOS
Registrado por: Mistral
Respuestas que lo sostienen: ChatGPT, Gemini, Kimi, Perplexity

> coinciden en que Makepad cubre las cinco plataformas con backends propios (Robrix y Moly como pruebas públicas) pero no trae abstracción de webview: la integración nativa debe escribirse por plataforma

**H80** — CONVERGENCIA · HECHOS
Registrado por: Mistral
Respuestas que lo sostienen: ChatGPT, Gemini, Claude, Kimi

> coinciden en que Qt en Linux obliga a Qt WebEngine (Chromium empaquetado), lo que incumple la prioridad de tamaño; en móvil usa el webview nativo pero con LGPLv3/comercial, C++ en el camino y fricción de FFI

**H81** — CONVERGENCIA · CONCLUSIONES
Registrado por: Mistral
Respuestas que lo sostienen: ChatGPT, Gemini, Grok, Kimi, Perplexity

> coinciden en que egui e iced igualan a Slint en rendimiento de escritorio pero no resuelven el

> multi-webview ni ofrecen un camino móvil oficial completo

**H82** — CONVERGENCIA · CONCLUSIONES
Registrado por: Mistral
Respuestas que lo sostienen: Gemini, Grok, Kimi, Perplexity

> coinciden en descartar GPUI para un producto de cinco plataformas: escritorio prioritario, móvil solo comunitario/experimental (gpui-mobile) y composición de webviews aún emergente

**H83** — CONVERGENCIA · HECHOS
Registrado por: Mistral
Respuestas que lo sostienen: Kimi, Perplexity

> coinciden en excluir CEF y motores empaquetados afines porque traen su propio Chromium: sin Android/iOS oficial e incompatibles con la prioridad de RAM y tamaño

**H84** — CONVERGENCIA · CONCLUSIONES
Registrado por: Mistral
Respuestas que lo sostienen: ChatGPT, Grok

> coinciden en que Fission declara cubrir las cinco plataformas (releases de 2026, Apache-2.0) pero ninguna encontró una API pública de webview nativo ni benchmarks; confianza baja en ambos casos

**H85** — CONVERGENCIA · FUENTES
Registrado por: Mistral
Respuestas que lo sostienen: Grok, Perplexity

> respaldan la comparativa de CPU de frameworks Rust en el mismo estudio goldstrikearch (2026), con Slint en 9,2 % y egui en 9,6 % de un núcleo, y Tauri por encima de ambos

**H86** — CONVERGENCIA · FUENTES
Registrado por: Mistral
Respuestas que lo sostienen: ChatGPT, Gemini, Grok, Kimi

> coinciden en advertir que los benchmarks de terceros usan perímetros distintos (app mínima con caché caliente frente a apps con contenido; binario con o sin librerías dinámicas), por lo

> que sus cifras difieren entre estudios sin contradecirse entre sí

**H119** — CONVERGENCIA · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT, Claude, Mistral, Kimi

> wry no permite colocar varios webviews hijos dentro de una misma pantalla en Android ni iOS; su soporte de build_as_child se limita a escritorio y, en Linux, a X11. operacion-P7-2026-11-09-2302.txt

**H120** — CONVERGENCIA · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT, Claude, Grok, Mistral, Kimi

> Tauri 2 añadió multi-ventana móvil mediante Activity Embedding en Android y UIScene en iOS alrededor de marzo de 2026, pero en teléfonos las ventanas se reemplazan entre sí en lugar de coexistir. operacion-P7-2026-11-09-2302.txt

**H121** — CONVERGENCIA · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT, Grok, Mistral, Kimi

> En Android, las instancias de WebView de una aplicación comparten un único proceso renderizador, por lo que diez instancias son técnicamente posibles, aunque exigen gestión explícita de memoria y ciclo de vida. operacion-P7-2026-11-09-2302.txt

**H122** — CONVERGENCIA · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT, Claude, Mistral, Kimi

> En iOS, cada WKWebView usa procesos WebContent separados y puede ser terminado por el sistema, lo que hace que diez instancias calientes sean mucho más riesgosas que en Android. operacion-P7-2026-11-09-2302.txt

**H123** — CONVERGENCIA · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT, Claude, Grok, Mistral, Kimi

> Ninguna de las respuestas encontró una medición pública que repitiera el caso exacto de diez webviews reales simultáneos y lo comparara con Slint y Flutter; las mediciones del usuario en Windows son el único dato directo de ese tipo. operacion-P7-2026-11-09-2302.txt

**H124** — CONVERGENCIA · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT, Grok, Mistral, Kimi

> Los benchmarks públicos de 2026 sitúan a Slint por debajo de Flutter en RAM, y a egui e iced en un rango de recursos cercano o competitivo con Slint en escritorio. operacion-P7-2026-11-09-2302.txt

**H125** — CONVERGENCIA · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT, Mistral, Kimi

> El benchmark de agosto de 2026 reporta a Tauri con una RAM mucho mayor que Slint cuando la webview es la interfaz: aproximadamente 309–348 MB frente a 106–124 MB en macOS. operacion-P7-2026-11-09-2302.txt

**H126** — CONVERGENCIA · FUENTES
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT, Mistral, Kimi

> Varias respuestas usan el mismo benchmark de 15 frameworks publicado en agosto de 2026 —Zenn— para comparar Slint, egui, iced, Flutter, Avalonia, Tauri y Compose Multiplatform. operacion-P7-2026-11-09-2302.txt

**H127** — CONVERGENCIA · CONCLUSIONES
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT, Claude, Mistral, Kimi

> Todas coinciden en que el trabajo crítico no es reemplazar Slint en escritorio, sino construir y validar una capa nativa de webviews para Android e iOS. operacion-P7-2026-11-09-2302.txt

**H128** — CONVERGENCIA · CONCLUSIONES
Registrado por: Perplexity
Respuestas que lo sostienen: Claude, Mistral, Kimi

> Las tres recomiendan un prototipo móvil en Kotlin y Swift que gestione varias vistas web, ejecute JavaScript, lea el DOM y permita adjuntar archivos antes de decidir la interfaz. operacion-P7-2026-11-09-2302.txt

**H129** — CONVERGENCIA · CONCLUSIONES
Registrado por: Perplexity
Respuestas que lo sostienen: Mistral, Kimi

> Ambas concluyen que, si el requisito es diez webviews simultáneos en un teléfono, el caparazón nativo con núcleo Rust es la única arquitectura realmente viable hoy; acepta sacrificar la unidad de la capa de UI. operacion-P7-2026-11-09-2302.txt

**H130** — CONVERGENCIA · CONCLUSIONES
Registrado por: Perplexity
Respuestas que lo sostienen: Mistral, Kimi

> Ambas proponen limitar o reciclar las vistas web en móvil mediante un pool de aproximadamente 2–4 instancias, pausando o desacoplando las no visibles. operacion-P7-2026-11-09-2302.txt

**H131** — CONVERGENCIA · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT, Mistral, Kimi

> Slint soporta oficialmente Android e iOS, pero ese soporte cubre la interfaz y no resuelve por sí solo la incrustación de múltiples webviews nativos en móvil. operacion-P7-2026-11-09-2302.txt

**H132** — CONVERGENCIA · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT, Claude, Mistral, Kimi

> Avalonia 12 ofrece NativeWebView en las cinco plataformas, pero su integración en Linux usa WPE WebKit con renderizado fuera de pantalla. operacion-P7-2026-11-09-2302.txt

**H133** — CONVERGENCIA · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT, Claude, Mistral, Kimi

> Qt WebView usa webviews nativos en Android, iOS, Windows y macOS, pero en Linux depende de Qt WebEngine, es decir, de Chromium empaquetado. operacion-P7-2026-11-09-2302.txt

**H134** — CONVERGENCIA · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT, Claude, Mistral, Kimi

> Compose Multiplatform es estable en iOS desde mayo de 2025, pero depende de la JVM en escritorio y de librerías de terceros para webviews de escritorio. operacion-P7-2026-11-09-2302.txt

**H135** — CONVERGENCIA · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: Grok, Mistral, Kimi

> Makepad cubre las cinco plataformas con una única base Rust, pero no incluye una integración de WebView nativa; requeriría un adaptador propio por plataforma. operacion-P7-2026-11-09-2302.txt

**H136** — CONVERGENCIA · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: Grok, Mistral, Kimi

> egui/eframe no ofrece una API documentada para incrustar webviews nativos, por lo que no resuelve directamente el requisito de ChatCouncil. operacion-P7-2026-11-09-2302.txt

**H137** — CONVERGENCIA · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: Grok, Mistral, Kimi

> iced carece de soporte móvil oficial y mantenido, por lo que ninguna de estas respuestas lo considera apto para las cinco plataformas. operacion-P7-2026-11-09-2302.txt

**H138** — CONVERGENCIA · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: Grok, Mistral, Kimi

> GPUI no está listo para producción en Android e iOS; su soporte móvil es experimental o de terceros. operacion-P7-2026-11-09-2302.txt

**H139** — CONVERGENCIA · FUENTES
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT, Claude, Mistral, Kimi

> Las respuestas citan documentación y código de wry 0.57.1 para concluir que los webviews hijos no están soportados en Android ni iOS. operacion-P7-2026-11-09-2302.txt

**H140** — CONVERGENCIA · FUENTES
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT, Claude, Mistral, Kimi

> Las respuestas citan la documentación oficial de Android sobre memoria y ciclo de vida de WebView, incluida la necesidad de WebView.destroy() y el manejo de onRenderProcessGone. operacion-P7-2026-11-09-2302.txt

**H141** — CONVERGENCIA · FUENTES
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT, Claude, Mistral, Kimi

> Las respuestas citan documentación de Apple/WebKit y análisis de terceros sobre WKProcessPool, procesos WebContent y terminación silenciosa de vistas. operacion-P7-2026-11-09-2302.txt

**H203** — CONVERGENCIA · HECHOS
Registrado por: Gemini
Respuestas que lo sostienen: ChatGPT, Claude, Mistral, Kimi

> Las cuatro coinciden en que wry solo soporta webviews hijos (WebViewBuilder::build_as_child) en escritorio (Windows, macOS y Linux sobre X11) y no tiene soporte para webviews hijos embebidos en Android ni iOS.

**H204** — CONVERGENCIA · HECHOS
Registrado por: Gemini
Respuestas que lo sostienen: ChatGPT, Mistral, Kimi, Perplexity

> Las cuatro coinciden en que el soporte de múltiples webviews hijos dentro de una sola ventana en Tauri v2 (Window::add_child / new Webview) está restringido a escritorio y requiere activar el feature flag unstable.

**H205** — CONVERGENCIA · HECHOS
Registrado por: Gemini
Respuestas que lo sostienen: ChatGPT, Claude, Grok, Mistral, Perplexity

> Las cinco coinciden en que en 2026 Tauri v2 incorporó soporte multi-ventana en móvil mediante Activity Embedding en Android y UIScene en iOS/iPadOS, donde en teléfonos las ventanas nuevas reemplazan o apilan la vista activa y solo coexisten lado a lado en tabletas e iPad.

**H206** — CONVERGENCIA · HECHOS
Registrado por: Gemini
Respuestas que lo sostienen: Claude, Mistral, Kimi

> Las tres coinciden en que Android ejecuta todos los WebView de una misma aplicación dentro de un único proceso renderizador compartido (salvo que se aíslen por perfiles, según P3), mientras que iOS crea un proceso WebContent independiente por cada WKWebView, elevando drásticamente el consumo de RAM.

**H207** — CONVERGENCIA · HECHOS
Registrado por: Gemini
Respuestas que lo sostienen: Mistral, Kimi

> Ambas coinciden en que en Android las instancias de WebView retienen memoria nativa fuera del heap de Java y exigen llamar explícitamente a WebView.destroy(), y que en iOS una sola página real en WKWebView puede consumir hasta ~200 MB de RAM.

**H208** — CONVERGENCIA · HECHOS
Registrado por: Gemini
Respuestas que lo sostienen: ChatGPT, Mistral, Kimi, Perplexity

> Las cuatro coinciden en que Makepad soporta las cinco plataformas (Windows, macOS, Linux, Android e iOS) con renderizado GPU propio y licencia permisiva (MIT o MIT/Apache-2.0), pero carece de widget o integración nativa de WebView y no cuenta con benchmarks públicos de RAM o arranque comparables con Slint.

**H209** — CONVERGENCIA · HECHOS
Registrado por: Gemini
Respuestas que lo sostienen: ChatGPT, Mistral, Kimi

> Las tres coinciden en que Makepad 1.0 se publicó en mayo de 2025 y citan la aplicación Robrix como evidencia de compilación real en las cinco plataformas.

**H210** — CONVERGENCIA · HECHOS
Registrado por: Gemini
Respuestas que lo sostienen: ChatGPT, Grok, Mistral, Kimi, Perplexity

> Las cinco coinciden en que egui / eframe (licencia MIT/Apache-2.0) carece de integración nativa para alojar múltiples webviews hijos.

**H211** — CONVERGENCIA · HECHOS
Registrado por: Gemini
Respuestas que lo sostienen: Grok, Mistral, Kimi, Perplexity

> Las cuatro coinciden en que Iced (licencia MIT, v0.14) es maduro en escritorio pero no tiene soporte móvil oficial (dependiendo de parches o esfuerzos comunitarios como iced_mobile) ni integración multi-webview.

**H212** — CONVERGENCIA · HECHOS
Registrado por: Gemini
Respuestas que lo sostienen: Grok, Mistral, Kimi, Perplexity

> Las cuatro coinciden en que GPUI (de Zed) está enfocado en escritorio y que su soporte para Android e iOS (gpui-mobile / experimental) es inmaduro para producción en las cinco plataformas.

**H213** — CONVERGENCIA · HECHOS
Registrado por: Gemini
Respuestas que lo sostienen: ChatGPT, Claude, Mistral, Kimi

> Las cuatro coinciden en que Qt 6 (Qt WebView) con bindings Rust (CXX-Qt, licencia MIT/Apache-2.0) utiliza los webviews nativos del sistema en Windows, macOS, Android e iOS, pero en Linux obliga a usar Qt WebEngine (Chromium empaquetado), y está sujeto a licencia LGPLv3/GPLv2 o comercial.

**H214** — CONVERGENCIA · HECHOS
Registrado por: Gemini
Respuestas que lo sostienen: ChatGPT, Claude, Kimi

> Las tres coinciden en que Avalonia ofrece el control NativeWebView en las cinco plataformas sobre los motores web del sistema, requiriendo FFI para conectar C#/.NET con el núcleo en Rust.

**H215** — CONVERGENCIA · HECHOS
Registrado por: Gemini
Respuestas que lo sostienen: ChatGPT, Claude, Mistral, Kimi

> Las cuatro coinciden en que Compose Multiplatform permite cubrir las cinco plataformas junto a un núcleo Rust por FFI/UniFFI, pero en escritorio corre sobre la JVM con un alto costo de memoria RAM frente a Slint.

**H216** — CONVERGENCIA · HECHOS
Registrado por: Gemini
Respuestas que lo sostienen: ChatGPT, Claude

> Ambas evalúan Uno Platform (C#/XAML, licencia Apache-2.0) con compilación Native AOT y soporte de WebView nativo en las cinco plataformas conectable a Rust vía FFI.

**H217** — CONVERGENCIA · HECHOS
Registrado por: Gemini
Respuestas que lo sostienen: ChatGPT, Grok

> Ambas evalúan el framework Rust Fission (v0.15 / 0.15.1 de octubre de 2026, licencia Apache-2.0, renderizado GPU sobre Vello/wgpu) y coinciden en que reclama soporte para las cinco plataformas pero carece de evidencia pública de multi-webview nativo y de benchmarks frente a Slint.

**H218** — CONVERGENCIA · HECHOS
Registrado por: Gemini
Respuestas que lo sostienen: Claude, Mistral

> Ambas evalúan Dioxus Native / Blitz y coinciden en descartarlo porque su soporte móvil es experimental/beta y carece de soporte de WebView nativo o motor JavaScript capaz de ejecutar las páginas de proveedores de IA.

**H219** — CONVERGENCIA · FUENTES
Registrado por: Gemini
Respuestas que lo sostienen: ChatGPT, Mistral, Kimi

> Las tres citan el mismo estudio comparativo de memoria de 15 frameworks en macOS publicado el 15 de agosto de 2026 (zenn.dev/mizugeeks/articles/1019cf2353d343) para contrastar el consumo de RAM de Slint frente a Flutter, Tauri, egui, Qt, Avalonia y Compose Multiplatform.

**H220** — CONVERGENCIA · FUENTES
Registrado por: Gemini
Respuestas que lo sostienen: Grok, Perplexity

> Ambas citan el estudio goldstrikearch.github.io/rust-gui-desktop-ecosystem-state (septiembre de 2026) como fuente de benchmarks de rendimiento de frameworks GUI en Rust.

**H221** — CONVERGENCIA · FUENTES
Registrado por: Gemini
Respuestas que lo sostienen: ChatGPT, Mistral

> Ambas identifican y citan el repositorio github.com/NucleusFramework/ComposeNativeWebview como la solución para incrustar webviews nativos del sistema en Compose Multiplatform.

**H222** — CONVERGENCIA · CONCLUSIONES
Registrado por: Gemini
Respuestas que lo sostienen: ChatGPT, Claude, Mistral, Kimi, Perplexity

> Las cinco coinciden en que Slint + wry debe mantenerse como base en escritorio y que resolver el caso móvil con vistas nativas reales exige una capa o contenedor nativo por plataforma (Kotlin en Android y Swift/UIKit en iOS) conectado al núcleo en Rust.

**H256** — CONVERGENCIA · HECHOS
Registrado por: Claude
Respuestas que lo sostienen: Grok, Perplexity, Kimi, Mistral, Gemini

> wry no soporta build_as_child (webviews hijos dentro de una ventana) en Android ni iOS; las cinco respuestas que lo tratan lo confirman con referencia a documentación o issues oficiales de wry/Tauri

**H257** — CONVERGENCIA · HECHOS
Registrado por: Claude
Respuestas que lo sostienen: Grok, Perplexity, ChatGPT, Kimi, Mistral, Gemini

> las seis coinciden en que no existen benchmarks públicos que reproduzcan la prueba del usuario con 10 webviews simultáneos cargando páginas reales; las mediciones propias del usuario en Windows son las únicas de esa clase

**H258** — CONVERGENCIA · HECHOS
Registrado por: Claude
Respuestas que lo sostienen: Grok, Perplexity, Kimi, Mistral, Gemini

> egui carece de integración nativa para incrustar webviews; las cinco respuestas que evalúan egui lo afirman

**H259** — CONVERGENCIA · HECHOS
Registrado por: Claude
Respuestas que lo sostienen: Grok, Perplexity, Kimi, Mistral, Gemini

> iced no tiene soporte móvil oficial; P7 y P5 citan el issue 302 donde el mantenedor declara que no le interesa soportar móviles, y P4 y P6 lo confirman

**H260** — CONVERGENCIA · HECHOS
Registrado por: Claude
Respuestas que lo sostienen: Perplexity, Kimi, Mistral, Gemini

> Makepad 1.0 (mayo 2025) no incluye widget ni abstracción de webview; la integración de WebView2/WKWebView/Android WebView sería trabajo propio

**H261** — CONVERGENCIA · HECHOS
Registrado por: Claude
Respuestas que lo sostienen: Grok, Perplexity, Kimi, Mistral, Gemini

> GPUI (Zed) no tiene soporte móvil de producción; las cinco que lo mencionan lo califican como experimental, comunitario o no viable para cinco plataformas

**H262** — CONVERGENCIA · HECHOS
Registrado por: Claude
Respuestas que lo sostienen: Kimi, Mistral, Gemini

> en Android, todas las instancias de android.webkit.WebView de una app comparten un único proceso renderizador aislado (SandboxedProcessService0), a diferencia de Chrome

**H263** — CONVERGENCIA · HECHOS
Registrado por: Claude
Respuestas que lo sostienen: Kimi, Mistral, Gemini

> en iOS, cada WKWebView lanza su propio proceso WebContent independiente, con costos de memoria reportados entre 50 y 200+ MB por instancia según la página cargada

**H264** — CONVERGENCIA · HECHOS
Registrado por: Claude
Respuestas que lo sostienen: Grok, Perplexity, Mistral

> Tauri 2 añadió soporte de múltiples ventanas en móvil mediante Activity Embedding en Android y UIScene en iOS, pero esto crea ventanas/escenas separadas, no una grilla de webviews dentro de una misma pantalla

**H265** — CONVERGENCIA · HECHOS
Registrado por: Claude
Respuestas que lo sostienen: Kimi, Gemini

> wry build_as_child funciona solo en X11 dentro de Linux; en Wayland requiere contenedor GTK (gtk::Fixed con build_gtk) o forzar XWayland

**H266** — CONVERGENCIA · CONCLUSIONES
Registrado por: Claude
Respuestas que lo sostienen: Grok, Perplexity, ChatGPT, Kimi, Mistral, Gemini

> las seis recomiendan mantener Slint como opción principal o como referencia para escritorio, dadas las mediciones existentes

**H267** — CONVERGENCIA · CONCLUSIONES
Registrado por: Claude
Respuestas que lo sostienen: Perplexity, Kimi, Mistral, Gemini

> el cuello de botella para diez webviews en móvil no es el toolkit de UI sino el modelo de procesos y memoria del SO (jetsam en iOS, renderer único en Android)

**H268** — CONVERGENCIA · CONCLUSIONES
Registrado por: Claude
Respuestas que lo sostienen: Grok, Perplexity, ChatGPT, Kimi, Mistral, Gemini

> las seis recomiendan como siguiente paso un prototipo que mida webviews nativos en dispositivos Android e iOS reales antes de decidir la arquitectura

**H310** — CONVERGENCIA · HECHOS
Registrado por: Kimi
Respuestas que lo sostienen: ChatGPT, Gemini, Claude, Grok, Mistral, Perplexity

> wry no soporta varios webviews hijos (build_as_child) en Android ni iOS; en móvil el modelo es un webview por actividad/ventana

**H311** — CONVERGENCIA · HECHOS
Registrado por: Kimi
Respuestas que lo sostienen: ChatGPT, Gemini, Claude, Mistral

> P2 no,P7|el "multi-window" móvil de Tauri 2.11 (Activity Embedding en Android, UIScene en iOS, 2026) crea ventanas/escenas separadas, no una grilla de diez webviews en una misma pantalla

**H312** — CONVERGENCIA · HECHOS
Registrado por: Kimi
Respuestas que lo sostienen: Gemini, Claude, Mistral

> Android comparte un único proceso renderizador aislado para todos los WebViews de la app; iOS da a cada WKWebView su propio proceso WebContent y jetsam los mata bajo presión de memoria

**H313** — CONVERGENCIA · HECHOS
Registrado por: Kimi
Respuestas que lo sostienen: Gemini, Claude, Mistral

> costos reportados por instancia de WKWebView del orden de 50-200/250 MB, lo que hace diez instancias calientes inviables en iPhone y obliga a agrupar/rotar instancias

**H314** — CONVERGENCIA · HECHOS
Registrado por: Kimi

Respuestas que lo sostienen: ChatGPT, Gemini, Grok, Mistral

> egui/eframe no tiene API documentada de incrustación de webviews; desktop sólido, iOS no oficial, licencia MIT/Apache

**H315** — CONVERGENCIA · HECHOS
Registrado por: Kimi
Respuestas que lo sostienen: ChatGPT, Gemini, Grok, Mistral

> Makepad 1.0 (13 may 2025) cubre las cinco plataformas con backends propios pero no trae webview; habría que integrar WebView2/WKWebView/Android WebView manualmente

**H316** — CONVERGENCIA · HECHOS
Registrado por: Kimi
Respuestas que lo sostienen: Gemini, Grok, Mistral

> iced no tiene camino móvil oficial (issue #302) ni integración de webviews

**H317** — CONVERGENCIA · HECHOS
Registrado por: Kimi
Respuestas que lo sostienen: Grok, Mistral

> Xilem está en estado demo-grade y no es base de producto hoy

**H318** — CONVERGENCIA · HECHOS
Registrado por: Kimi
Respuestas que lo sostienen: Gemini, Claude, Mistral

> Qt WebView usa webviews nativos en Windows/macOS/Android/iOS pero en Linux depende de QtWebEngine (Chromium empaquetado)

**H319** — CONVERGENCIA · HECHOS
Registrado por: Kimi
Respuestas que lo sostienen: ChatGPT, Gemini, Claude

> Compose Multiplatform para iOS es estable desde 1.8.0 (may 2025); ComposeNativeWebView usa AndroidView/UIKitView en móvil y wry/tao en desktop

**H320** — CONVERGENCIA · HECHOS
Registrado por: Kimi
Respuestas que lo sostienen: ChatGPT, Gemini, Grok

> Fission reclama las cinco plataformas con licencia Apache-2.0 pero es joven, sin API pública de multi-webview ni benchmarks

**H321** — CONVERGENCIA · HECHOS
Registrado por: Kimi
Respuestas que lo sostienen: ChatGPT, Grok

> GPUI tiene móvil solo vía gpui-mobile comunitario, sin multi-webview

**H322** — CONVERGENCIA · HECHOS
Registrado por: Kimi
Respuestas que lo sostienen: Gemini, Grok

> Freya no soporta webview en móvil (su capa webview es solo desktop)

**H323** — CONVERGENCIA · CONCLUSIONES
Registrado por: Kimi
Respuestas que lo sostienen: ChatGPT, Gemini, Claude, Mistral, Perplexity

> el cuello de botella es el alojamiento móvil de los webviews, no el toolkit de UI; todas recomiendan un prototipo en dispositivos físicos Android/iOS con las diez páginas reales antes de decidir

**H324** — CONVERGENCIA · CONCLUSIONES
Registrado por: Kimi
Respuestas que lo sostienen: Gemini, Claude, Mistral, Perplexity

> la única vía de primera clase hoy para diez webviews simultáneos en móvil es caparazón nativo (Kotlin/Swift) + núcleo Rust vía FFI/UniFFI, a costa de romper "un solo código" de UI

**H325** — CONVERGENCIA · CONCLUSIONES
Registrado por: Kimi
Respuestas que lo sostienen: ChatGPT, Gemini, Claude, Mistral

> no existen benchmarks públicos de diez webviews simultáneos con carga real; se necesitan mediciones propias con el mismo arnés

**H326** — CONVERGENCIA · FUENTES
Registrado por: Kimi
Respuestas que lo sostienen: Gemini, Mistral

> ambas citan el benchmark Zenn de mizugeeks (ago 2026, 15 frameworks) y el caso Sarrade (23 sep 2026) de 135 MB con egui a 30 MB con Slint

**H21** — AUSENCIA · HECHOS
Registrado por: Grok

> ninguna entrega medición propia o pública reproducible de arranque/RAM/CPU con exactamente diez webviews de proveedores reales en Android e iOS bajo las opciones propuestas

**H22** — AUSENCIA · HECHOS
Registrado por: Grok

> ninguna aporta cifras de tamaño de instalador o arranque en frío medidos en las cinco plataformas para la misma carga de diez webviews

**H62** — AUSENCIA · HECHOS
Registrado por: ChatGPT

> Ninguna aporta un benchmark de una alternativa frente a Slint y Flutter con el mismo hardware, las mismas diez páginas y las cuatro métricas solicitadas de RAM, CPU, arranque y tamaño.

**H63** — AUSENCIA · HECHOS
Registrado por: ChatGPT

> Ninguna demuestra en un teléfono y una tableta de cada plataforma móvil que una misma opción sostenga diez páginas reales autenticadas y operativas en paralelo, con resultados por dispositivo.

**H64** — AUSENCIA · HECHOS
Registrado por: ChatGPT

> Ninguna demuestra el flujo completo de ChatCouncil en una alternativa móvil con diez proveedores simultáneos, desde conservar la sesión y escribir en el DOM hasta adjuntar archivos y extraer respuestas.

**H113** — AUSENCIA · HECHOS
Registrado por: Mistral

> ninguna entrega rendimiento medido de las opciones propuestas bajo la carga real de diez webviews: todas remiten a fabricantes o a benchmarks con workloads distintos, y la comparación pedida contra Slint y Flutter solo existe en las mediciones de Windows del usuario

**H198** — AUSENCIA · HECHOS
Registrado por: Perplexity

> Ninguna respuesta entrega una medición pública que reproduzca exactamente diez proveedores de IA reales, con sesiones, DOM, adjuntos y redibujados, en Android o iOS. operacion-P7-2026-11-09-2302.txt

**H199** — AUSENCIA · HECHOS
Registrado por: Perplexity

> Ninguna respuesta entrega una comparación pública directa de las seis opciones candidatas bajo el mismo arnés de diez webviews, las cuatro métricas pedidas y las cinco plataformas. operacion-P7-2026-11-09-2302.txt

**H252** — AUSENCIA · HECHOS
Registrado por: Gemini

> Ninguna de las seis respuestas aporta mediciones empíricas de rendimiento (RAM, CPU, arranque o tamaño de binario) con diez webviews nativos simultáneos para ninguna de las opciones alternativas propuestas.

**H254** — AUSENCIA · HECHOS
Registrado por: Gemini

> Ninguna de las seis respuestas presenta pruebas verificadas de diez webviews nativos simultáneos funcionando con sesiones autenticadas, manipulación de DOM y carga de archivos adjuntos en Android e iOS para ninguno de los frameworks candidatos.

**H351** — AUSENCIA · HECHOS
Registrado por: Kimi

> ninguna respuesta ejecutó código ni midió ninguna alternativa: no hay mediciones propias con diez webviews reales en Android/iOS (todas lo declaran como límite)

**H23** — AUSENCIA · FUENTES
Registrado por: Grok

> ninguna cita código o test ejecutable propio que demuestre multi-webview hijo funcional en móvil bajo wry o equivalente Rust puro

**H114** — AUSENCIA · HECHOS
Registrado por: Mistral

> ninguna trae mediciones en tableta (iPad o Android), pese a que la pregunta la incluye; solo P4 cita documentación de uso side-by-side en tabletas, sin cifras

**H115** — AUSENCIA · HECHOS
Registrado por: Mistral

> ninguna mide el coste del flujo de adjuntos de archivos por opción; P4 declara expresamente que no se midió y P1 solo propone separarlo como prueba futura

**H116** — AUSENCIA · HECHOS
Registrado por: Mistral

> ninguna aporta la huella total atribuible bajo carga (procesos de WebView2, WKWebView y renderizadores incluidos), ni siquiera para las líneas base Slint y Flutter; P1 la señala como falta pendiente

**H305** — AUSENCIA · HECHOS
Registrado por: Claude

> ninguna respuesta mide ni compara el overhead de lectura del DOM y subida de archivos (las dos operaciones centrales de ChatCouncil) a través de las distintas opciones de webview

**H307** — AUSENCIA · HECHOS
Registrado por: Claude

> ninguna respuesta compara el consumo de batería o energía de las opciones evaluadas, a pesar de ser relevante para móvil

**H306** — AUSENCIA · HECHOS
Registrado por: Claude

> ninguna respuesta analiza sistemáticamente qué proveedores de IA bloquean o restringen el login en webviews embebidos; solo P1 menciona el caso de OAuth de Google como ejemplo aislado

**H308** — AUSENCIA · HECHOS
Registrado por: Claude

> ninguna respuesta evalúa la accesibilidad (a11y) de forma sistemática como criterio de comparación entre opciones; solo P7, P6 y P2 la mencionan al pasar como limitación de Makepad o egui

**H202** — AUSENCIA · CONCLUSIONES
Registrado por: Perplexity

> Ninguna respuesta entrega una recomendación final única y común sobre qué opción adoptar; las conclusiones operativas difieren entre conservar Slint más Native Shell, construir un caparazón nativo, o mantener Tauri como líder provisional. operacion-P7-2026-11-09-2302.txt

**H253** — AUSENCIA · HECHOS
Registrado por: Gemini

> Ninguna de las seis respuestas aporta cifras medidas de RAM, CPU, tiempo de arranque o tamaño de binario para Makepad ni para Fission que permitan compararlos cuantitativamente contra Slint y Flutter.

**H200** — AUSENCIA · HECHOS
Registrado por: Perplexity

> Ninguna respuesta verifica experimentalmente que Native Shell, Avalonia NativeWebView, Uno, Qt WebView o ComposeNativeWebView mantengan diez instancias activas con sesiones y adjuntos en Android e iOS. operacion-P7-2026-11-09-2302.txt

**H201** — AUSENCIA · FUENTES
Registrado por: Perplexity

> Ninguna respuesta encuentra una fuente primaria de Apple que cuantifique el límite exacto de procesos WebContent simultáneos por aplicación o el consumo por WKWebView en el caso de ChatCouncil. operacion-P7-2026-11-09-2302.txt

**H352** — AUSENCIA · HECHOS
Registrado por: Kimi

> ninguna trae mediciones en Linux con webviews embebidos (X11/Wayland); las únicas cifras Linux son de interfaz sola

**H353** — AUSENCIA · HECHOS
Registrado por: Kimi

> ninguna respuesta verificó el flujo completo de adjuntar archivos ni la lectura/escritura DOM en las alternativas evaluadas; solo aparece como prueba propuesta a futuro

**H304** — AUSENCIA · HECHOS
Registrado por: Claude

> ninguna respuesta entrega mediciones propias ejecutadas de rendimiento con 10 webviews en Android o iOS; todas reconocen que estos datos no existen públicamente

## Hallazgos no referenciados

**H13** — TENSION · CONCLUSIONES
Registrado por: Grok
Respuestas que lo sostienen: Perplexity, Kimi, Mistral, Gemini

> coinciden en que diez webviews calientes simultáneos en teléfono son inviables por el SO, pero P7/P6 recomiendan pool + pausa de JS como diseño viable mientras P2 enfatiza que incluso con pool el requisito de “diez vivos” puede fallar por OOM/Jetsam

**H14** — TENSION · CONCLUSIONES
Registrado por: Grok
Respuestas que lo sostienen: ChatGPT, Kimi

> ambos aceptan host nativo mínimo + núcleo Rust, pero P1 lo usa para decidir si el cuello está en la UI o en las páginas autenticadas, mientras P6 lo presenta como la única vía de primera clase si el requisito de diez en pantalla es estricto

**H15** — SINGULARIDAD · HECHOS
Registrado por: Grok
Respuestas que lo sostienen: ChatGPT

> única que introduce y detalla Native Shell (longbridge/nsh) 0.4.0 con fechas de release y backends por plataforma

**H16** — SINGULARIDAD · HECHOS
Registrado por: Grok
Respuestas que lo sostienen: Kimi

> única que aporta tabla de RAM de 15 frameworks (ago 2026) con cifras concretas para soak_gallery/sequence/workbench de Slint, egui, iced, Flutter, Avalonia, Tauri, Compose

**H17** — SINGULARIDAD · HECHOS
Registrado por: Grok
Respuestas que lo sostienen: Gemini

> única que detalla el requisito de puente JNI/ObjC de ~350-600 líneas y el diseño concreto de MultiWebViewManager con tres backends

**H18** — SINGULARIDAD · CONCLUSIONES
Registrado por: Grok
Respuestas que lo sostienen: Perplexity

> única que plantea explícitamente híbrido Tauri-solo-para-webviews + Slint-shell como

> arquitectura posible aunque probablemente peor en arranque/RAM

**H19** — SINGULARIDAD · FUENTES
Registrado por: Grok
Respuestas que lo sostienen: ChatGPT

> única que cita nsh.dev/docs y el informe de rendimiento del propio Native Shell

**H20** — SINGULARIDAD · FUENTES
Registrado por: Grok
Respuestas que lo sostienen: Kimi

> única que cita el estudio GoldStrikeArch/rust-gui y el benchmark de escalado de widgets (2026)

**H47** — TENSION · CONCLUSIONES
Registrado por: ChatGPT
Respuestas que lo sostienen: Grok, Perplexity

> Ambas reconocen la multiventana móvil de Tauri mediante actividades o escenas. P4 la toma como base para poner Tauri primero entre las alternativas. P7 mantiene Slint como UI principal y limita el papel de Tauri a una posible capa de webviews.

**H48** — TENSION · CONCLUSIONES
Registrado por: ChatGPT
Respuestas que lo sostienen: Gemini, Mistral

> Ambas identifican ComposeNativeWebview como una integración existente sobre wry y UniFFI. P2 lo presenta como una vía lista para diez instancias móviles. P5 lo deja como candidato por probar, con iOS y la licencia pendientes de verificación.

**H49** — TENSION · CONCLUSIONES
Registrado por: ChatGPT
Respuestas que lo sostienen: Gemini, Kimi, Perplexity

> Los tres coinciden en que egui requiere integración propia de webviews. P2 lo considera un sustituto posible de Slint si pesan la compilación o la atribución. P6 y P7 mantienen Slint porque no ven una ventaja que justifique el cambio.

**H50** — TENSION · CONCLUSIONES
Registrado por: ChatGPT
Respuestas que lo sostienen: Claude, Kimi

> Ambas prevén presión de memoria de WKWebView en iPhone. P3 propone usar diez proveedores reales en el prototipo y decidir según su supervivencia. P6 recomienda diseñar desde el comienzo para un pool de dos a cuatro vistas vivas.

**H51** — SINGULARIDAD · HECHOS
Registrado por: ChatGPT
Respuestas que lo sostienen: Claude

> Solo P3 detalla, a partir del código de wry 0.57.1, que Android instala el webview como contenido de la Activity y que la ruta iOS añade una subvista que toma el tamaño de la vista padre.

**H52** — SINGULARIDAD · HECHOS
Registrado por: ChatGPT
Respuestas que lo sostienen: Claude

> Solo P3 identifica los perfiles de WebView de Android como vía para separar renderizadores y precisa que cada vista afectada debe manejar onRenderProcessGone.

**H53** — SINGULARIDAD · HECHOS
Registrado por: ChatGPT
Respuestas que lo sostienen: Claude

> Solo P3 evalúa Uno Platform y registra cifras del fabricante para arranque y RAM en Windows y Android. También incluye Lynx y lo descarta por falta de Linux.

**H54** — SINGULARIDAD · HECHOS
Registrado por: ChatGPT
Respuestas que lo sostienen: Grok

> Solo P4 desarrolla Fission como candidato y distingue su cobertura declarada de plataformas de la falta de evidencia sobre webviews nativos múltiples.

**H55** — SINGULARIDAD · HECHOS
Registrado por: ChatGPT

Respuestas que lo sostienen: Mistral

> Solo P5 describe Tauri 3 alfa con runtime CEF de escritorio y menciona el estado de publicación del runtime experimental Servo/Verso.

**H56** — SINGULARIDAD · FUENTES
Registrado por: ChatGPT
Respuestas que lo sostienen: Mistral

> Solo P5 cita el caso de Sarrade del 23 de septiembre de 2026, que reporta un cambio de aproximadamente 135 MB con egui a 30 MB con Slint y su renderizador de software en una misma app.

**H57** — SINGULARIDAD · FUENTES
Registrado por: ChatGPT
Respuestas que lo sostienen: Kimi

> Solo P6 cita el ensayo de arranque de 2023 que reporta aproximadamente 230 ms para iced, 280 ms para egui y 380 ms para Tauri, con una carga distinta de ChatCouncil.

**H58** — SINGULARIDAD · FUENTES
Registrado por: ChatGPT
Respuestas que lo sostienen: Gemini

> Solo P2 usa el issue #4082 de Slint para comparar tiempos y RAM de compilación en una aplicación de 154 pantallas.

**H59** — SINGULARIDAD · FUENTES
Registrado por: ChatGPT
Respuestas que lo sostienen: Mistral

> Solo P5 cita el ensayo de Cross Mini App del 15 de octubre de 2026 que reporta la terminación de una prueba con diez webviews activos en iPhone.

**H60** — SINGULARIDAD · CONCLUSIONES
Registrado por: ChatGPT
Respuestas que lo sostienen: Gemini

> Solo P2 propone un contrato concreto de cinco operaciones para un gestor Rust de webviews y distingue los backends de escritorio, Linux/GTK y móvil.

**H61** — SINGULARIDAD · CONCLUSIONES
Registrado por: ChatGPT
Respuestas que lo sostienen: Perplexity

> Solo P7 propone como primera contingencia combinar Slint como interfaz con Tauri como capa de orquestación de webviews, y explicita el costo probable de combinar ambos.

**H65** — AUSENCIA · FUENTES
Registrado por: ChatGPT

> Ninguna aporta una prueba reproducible que mida si diez webviews ocultos conservan sesiones, conexiones y generación de respuestas en Android e iOS mientras solo uno o dos permanecen visibles.

**H66** — LIMITACION:AMBIGUEDAD
Registrado por: ChatGPT
Respuestas que lo sostienen: Claude, Grok, Mistral, Perplexity

> Las fechas de marzo y abril de 2026 para Tauri se refieren a hitos distintos, como el PR y la publicación de tauri-runtime-wry 2.11.0. El corpus tampoco usa de forma uniforme «ventana móvil» al describir pantallas completas y ventanas que coexisten en tabletas.

**H67** — LIMITACION:AMBIGUEDAD
Registrado por: ChatGPT
Respuestas que lo sostienen: Gemini, Claude, Kimi

> Los pasajes sobre licencia de Avalonia remiten a documentación y paquetes distintos. P3 enlaza un paquete 11.3.15, por lo que su referencia no resuelve por sí sola la discrepancia sobre el control de Avalonia 12.

**H68** — LIMITACION:AMBIGUEDAD
Registrado por: ChatGPT
Respuestas que lo sostienen: Gemini, Mistral, Kimi, Perplexity

> P2 y P5 describen egui/eframe como utilizable en iOS. P6 y P7 discuten su condición de plataforma oficialmente soportada. Los pasajes no establecen que las cuatro estén evaluando

> el mismo tipo de soporte.

**H69** — LIMITACION:AMBIGUEDAD
Registrado por: ChatGPT
Respuestas que lo sostienen: Gemini, Grok, Mistral, Kimi, Perplexity

> Las cifras de RAM y CPU para Tauri, egui y Slint mezclan proceso anfitrión, procesos auxiliares, reposo, picos y cargas distintas. El corpus no permite convertirlas en un orden de rendimiento para diez webviews.

**H70** — LIMITACION:OTRA
Registrado por: ChatGPT
Respuestas que lo sostienen: Gemini

> El adjunto conserva el aviso de que el elemento interactivo no se pudo cargar. La tabla textual está disponible, pero no pude determinar si el elemento añadía datos.

**H93** — TENSION · CONCLUSIONES
Registrado por: Mistral
Respuestas que lo sostienen: ChatGPT, Gemini, Grok, Kimi, Perplexity

> ante los mismos hechos sobre Tauri (multi-ventana móvil existente; webviews hijos no soportados en móvil; API inestable en escritorio), P4 lo recomienda como vía principal, P1 y P2 lo limitan a capa de orquestación de webviews, y P6/P7 lo descartan como runtime

**H94** — TENSION · CONCLUSIONES
Registrado por: Mistral
Respuestas que lo sostienen: ChatGPT, Gemini, Claude, Kimi

> ante Avalonia NativeWebView en cinco plataformas con coste .NET medido de 2-3 veces la RAM de Slint, P3 la llama la opción no considerada más completa, P6 la única no-Rust que cumple los requisitos y digna de prototipo, P1 la lista como viable, y P2 la descarta por el renderizado offscreen en Linux y la licencia comercial

**H95** — TENSION · CONCLUSIONES
Registrado por: Mistral
Respuestas que lo sostienen: Gemini, Kimi

> ante el mismo hecho de que Compose Multiplatform corre sobre JVM con la peor RAM medida del estudio (252-652 MB), P2 lo presenta como la única solución ya empaquetada con diez webviews nativos en las cinco plataformas (Camino B) y P6 lo declara descartable dada la prioridad absoluta de RAM

**H96** — TENSION · CONCLUSIONES
Registrado por: Mistral
Respuestas que lo sostienen: ChatGPT, Gemini, Claude, Kimi

> coinciden en los límites de memoria de iOS y Android para diez vistas, pero P6 concluye que diez simultáneas no sobreviven y prescribe un pool de 2-4, P2 asume que las mitigaciones (View.GONE, about:blank, pool rotativo) permiten mantener las diez con sesión, y P1/P3 concluyen que debe medirse en dispositivos físicos antes de decidir

**H97** — SINGULARIDAD · CONCLUSIONES
Registrado por: Mistral
Respuestas que lo sostienen: ChatGPT

> solo P1 propone Native Shell (longbridge/nsh 0.4.0, noviembre de 2026, Apache-2.0) como primera alternativa nueva a probar, con informe de rendimiento del fabricante (~14 MB privados, ~60 MB de working set); ninguna otra respuesta lo menciona

**H98** — SINGULARIDAD · CONCLUSIONES
Registrado por: Mistral
Respuestas que lo sostienen: Gemini

> solo P2 corrige la premisa del descarte de Dioxus: Dioxus Native (Blitz) renderiza HTML/CSS con Vello/wgpu sin usar webview, por lo que el motivo original del descarte no aplica a su motor nativo

**H99** — SINGULARIDAD · HECHOS
Registrado por: Mistral
Respuestas que lo sostienen: Gemini

> solo P2 aporta la crítica de costes de Slint: compilación de 42-271 s y rustc con 6,6-11 GB de RAM frente a 2-15 s de egui (issue #4082), y binarios de Slint que escalan hasta 87 MB en un caso industrial con 111 archivos .slint

**H100** — SINGULARIDAD · FUENTES
Registrado por: Mistral
Respuestas que lo sostienen: Claude

> solo P3 verificó leyendo el código fuente de wry 0.57.1 (android/mod.rs, main_pipe.rs, wkwebview/mod.rs) que en Android set_bounds no hace nada y hay un solo webview por Activity, y que en iOS el hijo toma el tamaño de la vista padre; las demás citan documentación o issues

**H101** — SINGULARIDAD · FUENTES
Registrado por: Mistral
Respuestas que lo sostienen: ChatGPT

> solo P1 cita guibench (23-jul-2026, app mínima con caché caliente: Slint 51,3 ms y 47,4 MiB; egui 58,6 ms; Flutter 126,8 ms; Tauri 163,9 ms) y el ensayo macOS de cuatro paneles (Slint 133 MB; Avalonia 361,3; Compose 652,6)

**H102** — SINGULARIDAD · FUENTES
Registrado por: Mistral
Respuestas que lo sostienen: Kimi

> solo P6 cita el estudio de 15 frameworks (15-ago-2026) con tabla por escenarios (Slint 111,8-127 MB; Flutter 165-205,7; Tauri 332,7-739; Compose 599,3-618,2) y el benchmark de escalado lavx (egui ~32 ms por edición frente a 4 µs de Slint)

**H103** — SINGULARIDAD · HECHOS
Registrado por: Mistral
Respuestas que lo sostienen: Gemini

> solo P2 menciona la reescritura experimental del cliente de Discord con GPUI (reportada el 5-nov-2026), el retardo de 300 ms a 3 s en la sincronización de cookies de WKWebsiteDataStore y la técnica de View.GONE para liberar buffers de GPU manteniendo vivo el contexto JS

**H104** — SINGULARIDAD · HECHOS
Registrado por: Mistral
Respuestas que lo sostienen: ChatGPT

> solo P1 advierte que iOS puede suspender el proceso en segundo plano (lo que cambia el alcance si se exige lectura del DOM en background) y que un proveedor restringe OAuth en

> navegadores embebidos, riesgo que persiste al cambiar de toolkit

**H105** — SINGULARIDAD · HECHOS
Registrado por: Mistral
Respuestas que lo sostienen: ChatGPT

> solo P1 detalla el flujo de adjuntos por plataforma (onShowFileChooser por vista en Android; selector personalizable vía WKUIDelegate desde iOS 18.4) y exige separar "el usuario selecciona el archivo" de "la app adjunta un archivo ya autorizado"

**H106** — SINGULARIDAD · HECHOS
Registrado por: Mistral
Respuestas que lo sostienen: ChatGPT

> solo P1 trae la medición del mantenedor de ComposeNativeWebView en un Mac M4 (90 fps frente a 90 fps con WKWebView directo) y la particularidad de que el puente se inyecta tras la carga en móvil frente a inicio de documento en escritorio

**H107** — SINGULARIDAD · HECHOS
Registrado por: Mistral
Respuestas que lo sostienen: ChatGPT

> solo P1 reporta WaterUI (0.4.1, sep-2026) con composición GPU por DMA-BUF en Linux sin lectura de vuelta a CPU; P4 la agrupa sin detalles entre las opciones inmaduras

**H108** — SINGULARIDAD · HECHOS
Registrado por: Mistral
Respuestas que lo sostienen: Claude

> solo P3 descarta Lynx por no soportar Linux (3.7, 16-abr-2026) y declara no haber podido abrir la fuente original de la cifra Tauri frente a Electron (142 frente a 149 MB) que llegó citada por un medio

**H109** — SINGULARIDAD · HECHOS
Registrado por: Mistral
Respuestas que lo sostienen: Kimi

> solo P6 cuantifica el coste de procesos de iOS ("un webview = 3 procesos, dos = 5, tres = 7; sin economías de escala") y la práctica de producción de no mantener más de 2 WKWebViews

> vivas

**H110** — SINGULARIDAD · HECHOS
Registrado por: Mistral
Respuestas que lo sostienen: Claude

> solo P3 documenta la limitación de z-order de Avalonia (las vistas nativas siempre quedan encima del contenido de Avalonia) y la fricción de wry con Activities de winit/GameActivity reportada por un usuario de foro

**H111** — SINGULARIDAD · CONCLUSIONES
Registrado por: Mistral
Respuestas que lo sostienen: Perplexity

> solo P7 plantea la interfaz nativa por plataforma con núcleo Rust compartido (WinUI 3, SwiftUI, GTK 4, Jetpack Compose) como plan B que elimina por completo el impuesto de renderizado del framework, aceptando romper el requisito de un solo código de UI

**H112** — SINGULARIDAD · FUENTES
Registrado por: Mistral
Respuestas que lo sostienen: Kimi

> solo P6 declara explícitamente que la skill research-protocol no existía en su entorno y que aplicó su espíritu (fuentes primarias fechadas, separación estricta entre rendimiento medido y afirmado)

**H117** — LIMITACION:CORPUS
Registrado por: Mistral
Respuestas que lo sostienen: Grok

> la tabla de fuentes de P4 está incompleta o dañada (celdas vacías y un "estudio independiente 2026" sin nombre), lo que impide verificar a qué fuente corresponde cada cifra concreta de ese bloque

**H118** — LIMITACION:CORPUS
Registrado por: Mistral
Respuestas que lo sostienen: Kimi

> P6 intercala fragmentos de registro de uso de herramientas y títulos ajenos (peticiones de razonamiento interno) que no forman parte de la respuesta sustantiva; los traté como metadato y no como contenido analizable

**H159** — TENSION · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: Grok, Mistral, Kimi

> Las tres coinciden en que Tauri 2 incorporó multi-ventana móvil en 2026, pero P4 lo interpreta como avance fuerte hacia el requisito, mientras P5 y P6 lo consideran insuficiente porque no crea una cuadrícula de diez webviews dentro de una pantalla de teléfono. operacion-P7-2026-11-09-2302.txt

**H160** — TENSION · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT, Mistral, Kimi

> Las tres reconocen que las ventanas nativas evitan la copia por textura observada en Flutter, pero P1 propone probar Native Shell antes de reescribir, mientras P5 y P6 recomiendan construir directamente un caparazón nativo Kotlin/Swift. operacion-P7-2026-11-09-2302.txt

**H161** — TENSION · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT, Claude, Mistral, Kimi

> Todas reconocen que Android permite varias instancias de WebView, pero P5 y P6 concluyen que diez simultáneas son viables solo con gestión estricta, mientras P1 y P3 las tratan como una hipótesis que debe validarse en dispositivos reales. operacion-P7-2026-11-09-2302.txt

**H162** — TENSION · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT, Claude, Mistral, Kimi

> Todas reconocen el riesgo de memoria en iOS, pero P5 y P6 ya lo consideran prácticamente invalidante para diez instancias calientes, mientras P1 y P3 proponen medirlo antes de descartar la arquitectura. operacion-P7-2026-11-09-2302.txt

**H163** — TENSION · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT, Claude, Grok, Mistral, Kimi

> Todas coinciden en que no hay benchmarks públicos equivalentes al caso de ChatCouncil, pero P1, P3 y P5 lo usan para priorizar pruebas propias, mientras P4 lo usa para mantener una recomendación provisional a favor de Tauri. operacion-P7-2026-11-09-2302.txt

**H164** — TENSION · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT, Claude, Kimi

> Las tres coinciden en que Avalonia es una alternativa a medir, pero P1 la presenta como opción viable con MIT, P3 como plan B y P6 como opción a prototipar solo si se tolera el coste de RAM y FFI de .NET. operacion-P7-2026-11-09-2302.txt

**H165** — TENSION · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT, Claude, Kimi

> Las tres coinciden en que Compose Multiplatform existe y es estable en iOS, pero P1 lo ve como una vía a probar con ComposeNativeWebView, P3 como opción con desventaja de JVM y P6 como candidata descartable por su mala RAM medida. operacion-P7-2026-11-09-2302.txt

**H166** — SINGULARIDAD · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT

> Solo P1 propone Native Shell (longbridge/nsh) como primera alternativa concreta a probar, con versión 0.4.0 del 4 de noviembre de 2026 y soporte de JavaScript y mensajería en las cinco plataformas. operacion-P7-2026-11-09-2302.txt

**H167** — SINGULARIDAD · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT

> Solo P1 evalúa BeeWare Toga, señalando su licencia BSD-3-Clause, versión 0.5.7 del 1 de octubre de 2026 y una brecha abierta en carga/descarga de archivos en WebView móvil.

> operacion-P7-2026-11-09-2302.txt

**H168** — SINGULARIDAD · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT

> Solo P1 menciona WaterUI, versión 0.4.1 del 11 de septiembre de 2026, con WebView y JavaScript, backends Apple/Android/Windows y rutas GTK4/Hydrolysis en Linux. operacion-P7-2026-11-09-2302.txt

**H169** — SINGULARIDAD · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT

> Solo P1 incluye la restricción de OAuth en navegadores embebidos como riesgo funcional que puede persistir independientemente del toolkit elegido. operacion-P7-2026-11-09-2302.txt

**H170** — SINGULARIDAD · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT

> Solo P1 propone separar la prueba de adjuntos en dos casos: archivo elegido por el usuario y archivo ya autorizado que ChatCouncil adjunta automáticamente. operacion-P7-2026-11-09-2302.txt

**H171** — SINGULARIDAD · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: Claude

> Solo P3 examina el código fuente de wry y afirma que en Android el webview ocupa toda la Activity y que set_bounds no hace nada; también que en iOS el webview hijo toma el tamaño de la vista padre. operacion-P7-2026-11-09-2302.txt

**H172** — SINGULARIDAD · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: Claude

> Solo P3 menciona Lynx como framework descartado porque soporta macOS y Windows pero

> no Linux. operacion-P7-2026-11-09-2302.txt

**H173** — SINGULARIDAD · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: Claude

> Solo P3 reporta que un usuario de winit con GameActivity no logró hacer funcionar wry en Android, aunque lo califica explícitamente como evidencia anecdótica de foro. operacion-P7-2026-11-09-2302.txt

**H174** — SINGULARIDAD · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: Grok

> Solo P4 evalúa Freya como opción Rust con UI GPU, multi-webview en escritorio y soporte Android parcial, pero sin webview móvil. operacion-P7-2026-11-09-2302.txt

**H175** — SINGULARIDAD · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: Grok

> Solo P4 menciona rust_widgets y Ribir entre opciones menores, descartándolas por falta de madurez o soporte móvil incompleto. operacion-P7-2026-11-09-2302.txt

**H176** — SINGULARIDAD · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: Grok

> Solo P4 cita los PR wry#1633 y tauri#14484, fusionados en marzo de 2026, como base del soporte de Activity Embedding y UIScene. operacion-P7-2026-11-09-2302.txt

**H177** — SINGULARIDAD · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: Mistral

> Solo P5 menciona explícitamente que Tauri 3.0 alfa añade un runtime CEF, pero solo para escritorio, con aumento de tamaño por Chromium y sin ecosistema de complementos. operacion-P7-2026-11-09-2302.txt

**H178** — SINGULARIDAD · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: Mistral

> Solo P5 reporta una prueba de estrés de Cross Mini App en la que diez WebViews activos en iPhone fueron terminados en torno a un segundo, con 50–80 MB por página en blanco y 120–200 MB por página real. operacion-P7-2026-11-09-2302.txt

**H179** — SINGULARIDAD · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: Mistral

> Solo P5 reporta el caso real de migración egui→Slint del 23 de septiembre de 2026, donde la misma aplicación pasó de aproximadamente 135 MB con egui a 30 MB con Slint y renderizador de software. operacion-P7-2026-11-09-2302.txt

**H180** — SINGULARIDAD · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: Kimi

> Solo P6 propone una arquitectura concreta de pool de 2–4 vistas con congelación de JavaScript, rehidratación bajo demanda y contenedor nativo en Kotlin/Swift. operacion-P7-2026-11-09-2302.txt

**H181** — SINGULARIDAD · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: Kimi

> Solo P6 menciona Floem, Vizia y Xilem como alternativas Rust descartadas por falta de móvil y multi-webview. operacion-P7-2026-11-09-2302.txt

**H182** — SINGULARIDAD · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: Kimi

> Solo P6 menciona explícitamente que Ultralight y CEF quedan fuera porque empaquetan motores propios y no reutilizan la sesión del navegador del sistema. operacion-P7-2026-11-09-2302.txt

**H183** — SINGULARIDAD · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: Kimi

> Solo P6 incluye el benchmark de escalado de septiembre–noviembre de 2026, donde Slint edita un valor en 4 µs, iced construye 39.023 widgets en 7,3 ms y egui requiere aproximadamente 32 ms para una edición pequeña. operacion-P7-2026-11-09-2302.txt

**H184** — SINGULARIDAD · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: Gemini

> Solo P2 presenta una tabla comparativa extensa que incluye estimaciones específicas de arranque, binario, RAM, CPU y estado de webviews para Slint, Flutter, egui/iced, Makepad, Dioxus Native/Blitz, ComposeNativeWebView, Tauri v2, Avalonia y Qt. operacion-P7-2026-11-09-2302.txt

**H185** — SINGULARIDAD · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: Gemini

> Solo P2 afirma que ComposeNativeWebView usa DirectComposition en Windows, NSView en macOS y WebKit2GTK en Linux, perforando un hueco transparente en la capa de Compose sin copiar texturas. operacion-P7-2026-11-09-2302.txt

**H186** — SINGULARIDAD · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: Gemini

> Solo P2 propone implementar un MultiWebViewManager en Rust con tres backends: wry hijo en Windows/macOS, gtk::Fixed con build_gtk en Linux, y módulos Kotlin/Swift de aproximadamente 350 líneas en Android/iOS. operacion-P7-2026-11-09-2302.txt

**H187** — SINGULARIDAD · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: Gemini

> Solo P2 menciona el problema de compilación de Slint: cambiar una línea de UI puede tardar 21–57 s y consumir 6,6–11 GB de RAM de rustc según el issue 4082. operacion-P7-2026-11-

> 09-2302.txt

**H188** — SINGULARIDAD · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: Gemini

> Solo P2 menciona la varianza del binario de Slint según plataforma y número de componentes, citando 19,81 MiB en Linux y hasta 87 MB antes de optimizaciones en un caso industrial. operacion-P7-2026-11-09-2302.txt

**H189** — SINGULARIDAD · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: Gemini

> Solo P2 propone mantener vistas Android desacopladas con View.GONE para liberar surface buffers de GPU conservando contexto JavaScript y conexiones SSE/WebSocket. operacion-P7-2026-11-09-2302.txt

**H190** — SINGULARIDAD · HECHOS
Registrado por: Perplexity
Respuestas que lo sostienen: Gemini

> Solo P2 menciona la latencia de sincronización de cookies de WKWebsiteDataStore.default.httpCookieStore a disco, de 300 ms a 3 s, como riesgo de sesión en iOS. operacion-P7-2026-11-09-2302.txt

**H191** — SINGULARIDAD · CONCLUSIONES
Registrado por: Perplexity
Respuestas que lo sostienen: ChatGPT

> Solo P1 concluye que la decisión inmediata debe ser conservar Slint en escritorio y enfrentar Native Shell contra un host móvil mínimo antes de reescribir la interfaz. operacion-P7-2026-11-09-2302.txt

**H192** — SINGULARIDAD · CONCLUSIONES
Registrado por: Perplexity
Respuestas que lo sostienen: Gemini

> Solo P2 concluye que, si la interfaz tendrá menos de 15 pantallas, Slint es excelente; si habrá decenas de vistas complejas o se quiere evitar la licencia con atribución y los tiempos de compilación, conviene probar egui/eframe con wry. operacion-P7-2026-11-09-2302.txt

**H193** — SINGULARIDAD · CONCLUSIONES
Registrado por: Perplexity
Respuestas que lo sostienen: Gemini

> Solo P2 sostiene que Tauri v2 podría reevaluarse porque varios webviews que comparten CoreWebView2Environment comparten procesos navegador, GPU y red, añadiendo solo 15–25 MB por renderizador ligero en vez de 122 MB. operacion-P7-2026-11-09-2302.txt

**H194** — SINGULARIDAD · CONCLUSIONES
Registrado por: Perplexity
Respuestas que lo sostienen: Grok

> Solo P4 concluye que Tauri 2 + wry lidera actualmente la viabilidad de multi-webview en las cinco plataformas, con confianza alta en cobertura de plataformas y moderada en rendimiento exacto. operacion-P7-2026-11-09-2302.txt

**H195** — SINGULARIDAD · CONCLUSIONES
Registrado por: Perplexity
Respuestas que lo sostienen: Grok

> Solo P4 propone prototipar Freya + wry multi-webview en escritorio y luego probar Android sin webview como vía de validación parcial. operacion-P7-2026-11-09-2302.txt

**H196** — SINGULARIDAD · CONCLUSIONES
Registrado por: Perplexity
Respuestas que lo sostienen: Mistral

> Solo P5 concluye que, si basta escritorio y tableta/iPad, Slint+wry —o Tauri 2.x si se acepta UI web— cubre el caso con el mejor perfil medido. operacion-P7-2026-11-09-2302.txt

**H197** — SINGULARIDAD · CONCLUSIONES
Registrado por: Perplexity
Respuestas que lo sostienen: Kimi

> Solo P6 concluye que el límite decisivo no es el toolkit de UI sino el modelo de composición de webviews nativos en móvil, que obliga a código nativo por plataforma y a un diseño de pool. operacion-P7-2026-11-09-2302.txt

**H233** — TENSION · CONCLUSIONES
Registrado por: Gemini
Respuestas que lo sostienen: Claude, Grok, Mistral, Kimi, Perplexity

> Coinciden en que el soporte móvil de Tauri 2 + wry funciona mediante ventanas/escenas separadas (Activity Embedding / UIScene) y no como múltiples webviews hijos en una vista, pero P4 recomienda Tauri 2 + wry como la opción #1 más viable para las 5 plataformas, mientras que P3, P5, P6 y P7 la descartan como solución de UI principal porque su propia interfaz es otro webview y no resuelve mostrar diez vistas en un teléfono.

**H234** — TENSION · CONCLUSIONES
Registrado por: Gemini
Respuestas que lo sostienen: Kimi, Perplexity

> Ambas coinciden en que la API de webviews hijos de Tauri/wry está marcada como unstable y limitada a escritorio, pero P7 propone como primera contingencia usar Tauri solo como orquestador de webviews bajo una UI Slint, mientras que P6 concluye que usar Tauri sin su capa HTML/JS no aporta nada frente a usar wry directamente y empeora la RAM.

**H235** — TENSION · CONCLUSIONES
Registrado por: Gemini
Respuestas que lo sostienen: ChatGPT, Claude, Mistral, Kimi, Perplexity

> A partir de las restricciones de memoria y procesos WebContent de WKWebView en iOS, P6 (y P5 para teléfonos iPhone) concluyen que diez webviews simultáneos en móvil son inviables y recomiendan cambiar la arquitectura a un pool de 2 a 4 instancias con congelación de JS y rehidratación, mientras que P1, P3 y P7 recomiendan construir un prototipo para probar las diez instancias activas simultáneas en dispositivos móviles reales (advirtiendo P1 que congelar vistas ocultas que esperan respuestas cuenta como fallo funcional).

**H236** — TENSION · CONCLUSIONES
Registrado por: Gemini
Respuestas que lo sostienen: ChatGPT, Mistral, Kimi

> Coinciden en los hechos sobre Qt 6 + CXX-Qt (buena RAM en escritorio, webviews nativos en 4 plataformas, pero Qt WebEngine/Chromium obligatorio en Linux y licencia LGPLv3/comercial), pero P1 y P5 lo mantienen como alternativa viable con costos, mientras que P6 recomienda descartarlo de plano (sumando que en Qt 6.12 WebEngine se separa y deja de distribuir binarios).

**H237** — TENSION · CONCLUSIONES
Registrado por: Gemini
Respuestas que lo sostienen: Mistral, Kimi, Perplexity

> Sobre el rendimiento de CPU de egui, P6 concluye a partir de su documentación que en reposo no gasta CPU y se comporta igual que Slint, mientras que P5 y P7 concluyen que su arquitectura de modo inmediato penaliza el CPU y la RAM al repintar toda la interfaz cuando hay actividad en pantalla.

**H238** — SINGULARIDAD · HECHOS
Registrado por: Gemini
Respuestas que lo sostienen: ChatGPT

> Solo P1 identifica y evalúa Native Shell (longbridge/nsh v0.4.0 del 4 de noviembre de 2026, licencia Apache-2.0) como framework Rust que envuelve los motores web nativos de las cinco plataformas con inyección JS, e incluye las mediciones del fabricante en Windows (~14 MB de memoria privada, ~60 MB de working set y binario de 15,0 MB).

**H239** — SINGULARIDAD · CONCLUSIONES
Registrado por: Gemini
Respuestas que lo sostienen: ChatGPT

> Solo P1 recomienda probar Native Shell (longbridge/nsh) en paralelo contra un host móvil mínimo antes de decidir cambios en la interfaz.

**H240** — SINGULARIDAD · HECHOS
Registrado por: Gemini
Respuestas que lo sostienen: ChatGPT

> Solo P1 evalúa WaterUI (0.4.1 del 11 de septiembre de 2026, MIT/Apache-2.0, con composición GPU por DMA-BUF sobre WPE en Linux) y BeeWare Toga (0.5.7 del 1 de octubre de 2026, BSD-3-Clause, Python + FFI Rust).

**H241** — SINGULARIDAD · FUENTES
Registrado por: Gemini
Respuestas que lo sostienen: ChatGPT

> Solo P1 cita el benchmark guibench (github.com/lumen-fx/guibench, 23 de julio de 2026, en Arch Linux con Weston headless) que compara arranque, RAM PSS y binario entre Slint 1.17.1 (51,3 ms / 47,4 MiB), egui 0.35.0 (58,6 ms / 48,4 MiB), Qt Widgets 6.11.1 (54,9 ms / 23,6 MiB), Flutter 3.44.7 (126,8 ms / 78,2 MiB) y Tauri 2.11.5 (163,9 ms / 59,0 MiB).

**H242** — SINGULARIDAD · HECHOS
Registrado por: Gemini
Respuestas que lo sostienen: ChatGPT

> Solo P1 aborda los mecanismos específicos de adjuntos de archivos en móvil (onShowFileChooser en Android y WKUIDelegate desde iOS 18.4), los bloqueos de OAuth en webviews embebidos, la reutilización de CoreWebView2Environment en Windows y el modo HCPP de Flutter 3.44 en Android API 34+.

**H243** — SINGULARIDAD · HECHOS
Registrado por: Gemini
Respuestas que lo sostienen: Claude

> Solo P3 inspecciona el código fuente de wry 0.57.1 (android/mod.rs, main_pipe.rs, wkwebview/mod.rs) y demuestra que en Android wry ocupa toda la Activity con un único webview, que set_bounds solo está implementado en macOS (no hace nada en Android ni iOS), y que wry requiere AppCompatActivity (WryActivity) mientras Slint en Android usa winit/GameActivity.

**H244** — SINGULARIDAD · HECHOS
Registrado por: Gemini
Respuestas que lo sostienen: Claude

> Solo P3 evalúa Crux (de Red Badger), Lynx 3.7 (16 de abril de 2026, sin soporte Linux) y el lanzamiento preliminar de Qt Bridge for Rust (27 de octubre de 2026).

**H245** — SINGULARIDAD · HECHOS
Registrado por: Gemini
Respuestas que lo sostienen: Grok

> Solo P4 menciona los proyectos rust_widgets y Ribir dentro del ecosistema de interfaces en Rust.

**H246** — SINGULARIDAD · HECHOS
Registrado por: Gemini
Respuestas que lo sostienen: Mistral

> Solo P5 documenta Tauri 3.0 (alfa desde septiembre/octubre de 2026 con runtime CEF en escritorio) y el estado del runtime experimental Servo/Verso, citando el análisis de issoh.co.jp del 31 de octubre de 2026.

**H247** — SINGULARIDAD · FUENTES
Registrado por: Gemini
Respuestas que lo sostienen: Mistral

> Solo P5 cita el caso de estudio de Trystan Sarrade (23 de septiembre de 2026, trystan-sarrade.com) donde migrar una misma aplicación de egui a Slint (con renderizador de software) redujo el uso de RAM de ~135 MB a ~30 MB.

**H248** — SINGULARIDAD · FUENTES
Registrado por: Gemini
Respuestas que lo sostienen: Mistral

> Solo P5 cita la prueba de estrés en iPhone del blog Cross Mini App (15 de octubre de 2026), según la cual diez instancias activas de WKWebView provocan que el sistema operativo termine el proceso por jetsam en ~1 segundo.

**H249** — SINGULARIDAD · FUENTES
Registrado por: Gemini
Respuestas que lo sostienen: Kimi

> Solo P6 cita el benchmark de escalado de UI compleja con 3.000 filas vivas (news.lavx.hu, sep–nov 2026), donde una edición cuesta 4 µs en Slint frente a 7,3 ms en Iced, 14 ms en Floem, 32 ms en egui y 46 ms en Vizia, y el benchmark de lukaskalbertodt.github.io (2023).

**H250** — SINGULARIDAD · HECHOS
Registrado por: Gemini
Respuestas que lo sostienen: Kimi

> Solo P6 detalla las condiciones de la licencia de Slint (gratuita bajo modalidad Royalty-free solo si se incluye atribución visible como AboutSlint, o GPLv3/comercial sin ella) y menciona las mejoras de Slint 1.15 (febrero de 2026, con puerto iOS financiado por NLnet).

**H251** — SINGULARIDAD · HECHOS
Registrado por: Gemini
Respuestas que lo sostienen: Perplexity

> Solo P7 incluye Relm4 / GTK como alternativa evaluada en su tabla comparativa.

**H255** — LIMITACION:CORPUS
Registrado por: Gemini
Respuestas que lo sostienen: ChatGPT, Claude

> El archivo adjunto contiene marcas de censura (■■■) que ocultan el nombre de un proveedor que restringe OAuth en navegadores embebidos en P1 y el nombre del sistema autor al inicio de P3, además de un fragmento truncado en el preámbulo de P3.

**H276** — TENSION · CONCLUSIONES
Registrado por: Claude
Respuestas que lo sostienen: Grok, Perplexity

> ambos reconocen que Tauri 2 es la opción más madura con multi-webview, pero P4 lo posiciona como alternativa principal viable, mientras P7 dice explícitamente “útil como capa de webview, no como UI principal” porque su UI en webview contradice la prioridad de rendimiento

**H277** — TENSION · CONCLUSIONES
Registrado por: Claude
Respuestas que lo sostienen: Perplexity, Kimi, Mistral

> coinciden en que diez webviews simultáneos no sobreviven en un teléfono, pero difieren en la estrategia: P6 recomienda pool de 2-4 con congelación de JS e hidratación bajo demanda; P5 dice que la única vía es caparazón nativo (Kotlin/Swift) con núcleo Rust; P7 dice que UI nativa por plataforma es “plan B si Slint falla”

**H278** — TENSION · CONCLUSIONES
Registrado por: Claude

Respuestas que lo sostienen: Kimi, Gemini

> ambos reconocen que egui y Slint empatan en RAM/binario en ciertos benchmarks, pero P6 dice que las alternativas Rust “no aportan nada que justifique migrar” y favorece el modelo lazy de Slint en UI grandes; P2 dice que egui o Makepad podrían reemplazar a Slint si se quieren evitar los tiempos de compilación de Slint (42-271s, issue #4082) y la licencia con atribución

**H279** — TENSION · CONCLUSIONES
Registrado por: Claude
Respuestas que lo sostienen: Grok, Mistral

> ambos reconocen a Tauri como la opción más madura para orquestación de webviews, pero P4 le da confianza alta en viabilidad multiplataforma, mientras P5 concluye que “la única vía de primera clase para el requisito móvil” es caparazón nativo + Rust (UniFFI), descartando Tauri para ese propósito

**H280** — TENSION · CONCLUSIONES
Registrado por: Claude
Respuestas que lo sostienen: ChatGPT, Perplexity

> ambos mencionan Makepad como opción interesante, pero P7 lo propone como la alternativa Rust “más interesante que no aparece en tu descarte” y le dedica análisis extenso; P1 lo menciona brevemente, condicionando su evaluación al mismo host nativo por plataforma que describe para las demás opciones

**H281** — TENSION · CONCLUSIONES
Registrado por: Claude
Respuestas que lo sostienen: Gemini, Kimi

> ambos citan las mismas fuentes sobre iOS WKWebView, pero P2 recomienda mantener 1-2 WKWebView visibles con 8-9 desacoplados del hardware view; P6 recomienda nunca tener más de 2 vivas y reutilizarlas, con pool de 2-4 activos

**H282** — SINGULARIDAD · HECHOS
Registrado por: Claude
Respuestas que lo sostienen: ChatGPT

> solo P1 identifica y propone Native Shell (longbridge/nsh 0.4.0, 4 nov 2026, Apache-2.0) como opción nueva con backends en las cinco plataformas y API de JavaScript/mensajería

**H283** — SINGULARIDAD · HECHOS
Registrado por: Claude
Respuestas que lo sostienen: ChatGPT

> solo P1 menciona Uno Platform (C#/XAML, versión 6.6 con Native AOT, julio 2026) como opción

**H284** — SINGULARIDAD · HECHOS
Registrado por: Claude
Respuestas que lo sostienen: ChatGPT

> solo P1 menciona BeeWare Toga (Python, BSD-3-Clause, 0.5.7) como opción, aunque la descarta por prioridad baja

**H285** — SINGULARIDAD · HECHOS
Registrado por: Claude
Respuestas que lo sostienen: ChatGPT

> solo P1 menciona WaterUI 0.4.1 (septiembre 2026) con WebView y JavaScript, backends Apple/Hydrolysis/GTK4 y composición GPU por DMA-BUF en Linux

**H286** — SINGULARIDAD · HECHOS
Registrado por: Claude
Respuestas que lo sostienen: ChatGPT

> solo P1 cita el benchmark guibench (lumen-fx, 23 jul 2026, Arch Linux, i9-12900K, Weston headless) con cifras de arranque: Slint 51,3 ms, egui 58,6 ms, Qt 54,9 ms, Flutter 126,8 ms, Tauri 163,9 ms

**H287** — SINGULARIDAD · HECHOS
Registrado por: Claude
Respuestas que lo sostienen: Gemini

> solo P2 cita el issue #4082 de Slint con datos de compilación medidos: 42-271 s y 6,6-11 GB de RAM en Slint 1.18.1 (con 154 pantallas/111 archivos .slint) frente a 2-15 s en egui

**H288** — SINGULARIDAD · HECHOS
Registrado por: Claude
Respuestas que lo sostienen: Gemini

> solo P2 describe el mecanismo interno de wry en Android: genera archivos Kotlin (WryActivity) que atan un único android.webkit.WebView a la AppCompatActivity principal

**H289** — SINGULARIDAD · HECHOS
Registrado por: Claude
Respuestas que lo sostienen: Gemini

> solo P2 menciona Crux (Red Badger) como opción y lo descarta porque exige un shell de UI nativo distinto por plataforma

**H290** — SINGULARIDAD · HECHOS
Registrado por: Claude
Respuestas que lo sostienen: Gemini

> solo P2 reporta que Discord estaría reescribiendo su app de escritorio con GPUI (reporte del 5 nov 2026)

**H291** — SINGULARIDAD · HECHOS
Registrado por: Claude
Respuestas que lo sostienen: Kimi

> solo P6 cita un benchmark de escalado (3000 filas vivas, sep-nov 2026) comparando edición incremental: Slint 4 µs, iced 7,3 ms, egui 32 ms (19 ms sin accesibilidad)

**H292** — SINGULARIDAD · HECHOS
Registrado por: Claude
Respuestas que lo sostienen: ChatGPT

> solo P1 cita la medición del mantenedor de ComposeNativeWebView: 90 fps con Canvas 2D y 31-34 fps con shader WebGL, comparando embebido vs WKWebView directo en Mac M4

**H293** — SINGULARIDAD · HECHOS
Registrado por: Claude
Respuestas que lo sostienen: ChatGPT

> solo P1 plantea que iOS UIKit puede suspender el proceso al pasar a segundo plano, cambiando el alcance de la solución si el requisito incluye leer DOM con la app suspendida

**H294** — SINGULARIDAD · HECHOS
Registrado por: Claude
Respuestas que lo sostienen: ChatGPT

> solo P1 señala que ciertos proveedores (ejemplo: Google) restringen OAuth en navegadores embebidos, lo cual persistiría independientemente del toolkit elegido

**H295** — SINGULARIDAD · CONCLUSIONES
Registrado por: Claude
Respuestas que lo sostienen: Gemini

> solo P2 cuantifica el puente nativo móvil en ~350 líneas de Kotlin (JNI) y ~350 líneas de Swift/ObjC para gestionar los 10 webviews, y presenta dos caminos arquitectónicos detallados (A: Slint/egui + puente propio; B: Compose Multiplatform + ComposeNativeWebView)

**H296** — SINGULARIDAD · CONCLUSIONES
Registrado por: Claude
Respuestas que lo sostienen: Gemini

> solo P2 posiciona a Compose Multiplatform + ComposeNativeWebView como “la única opción con un solo código de UI que hoy ofrece un componente ya hecho para múltiples webviews nativos sin copia por textura en las 5 plataformas”

**H297** — SINGULARIDAD · CONCLUSIONES
Registrado por: Claude
Respuestas que lo sostienen: ChatGPT

> solo P1 recomienda Native Shell como primera alternativa a probar, con una secuencia de validación que empieza por enfrentarlo al host móvil mínimo antes de reescribir la interfaz

**H298** — SINGULARIDAD · CONCLUSIONES
Registrado por: Claude
Respuestas que lo sostienen: Perplexity

> solo P7 señala explícitamente que “diez webviews nativos simultáneos en Android e iOS normalmente no significa diez ventanas del sistema” sino diez instancias embebidas como

> vistas hijas, y que la confusión entre ventanas y vistas es una corrección necesaria al planteamiento

**H299** — SINGULARIDAD · CONCLUSIONES
Registrado por: Claude
Respuestas que lo sostienen: Grok

> solo P4 identifica Fission como candidata digna de sección propia, con versión 0.15.1 (octubre 2026), cinco plataformas reclamadas y CLI de packaging, aunque sin evidencia de multi-webview ni benchmarks

**H300** — SINGULARIDAD · FUENTES
Registrado por: Claude
Respuestas que lo sostienen: ChatGPT

> solo P1 cita nsh.dev (documentación y reporte de rendimiento de Native Shell)

**H301** — SINGULARIDAD · FUENTES
Registrado por: Claude
Respuestas que lo sostienen: ChatGPT

> solo P1 cita el benchmark guibench de lumen-fx como fuente de comparación de arranque y RAM

**H302** — SINGULARIDAD · FUENTES
Registrado por: Claude
Respuestas que lo sostienen: Kimi

> solo P6 cita la fuente news.lavx.hu para el benchmark de escalado con 3000 filas

**H303** — SINGULARIDAD · FUENTES
Registrado por: Claude
Respuestas que lo sostienen: Gemini

> solo P2 cita el código fuente de Chromium (chromium.googlesource.com, android_webview/renderer/README.md) como fuente primaria del modelo de proceso renderizador único en Android

**H309** — LIMITACION:AMBIGUEDAD
Registrado por: Claude
Respuestas que lo sostienen: Grok, ChatGPT

> P4 cita “goldstrikearch Rust GUI ecosystem measured, 2026-09” y P1 cita un “ensayo público de macOS, del 15 ago 2026, una aplicación de cuatro paneles”; no puedo determinar con certeza si son la misma fuente o dos fuentes distintas porque las fechas y descripciones no coinciden exactamente, pero ambos citan goldstrikearch en sus listas de fuentes

**H332** — TENSION · HECHOS
Registrado por: Kimi
Respuestas que lo sostienen: ChatGPT, Gemini

> ambas describen la ruta WPE WebKit offscreen de Avalonia en Linux; P2 concluye que reintroduce el coste de copia de textura que mediste en Flutter, mientras P1 lo lista solo como riesgo por verificar ("trasladar la hipótesis de bajo coste de Windows a Linux")

**H333** — TENSION · CONCLUSIONES
Registrado por: Kimi
Respuestas que lo sostienen: Grok, Gemini, Mistral, Perplexity

> sobre Tauri 2: P4 lo rankea como la opción más madura y viable para las cinco plataformas; P2 y P7 lo restringen a capa de orquestación de webviews (su UI sería un webview más, choca con la prioridad) y P5 lo descarta para diez vistas en una sola pantalla móvil

**H334** — TENSION · CONCLUSIONES
Registrado por: Kimi
Respuestas que lo sostienen: ChatGPT, Gemini, Claude

> coinciden en que hace falta una capa móvil propia, pero difieren en el primer experimento: P1 prueba primero Native Shell manteniendo Slint como referencia; P2 mantiene Slint+wry y propone escribir los puentes JNI/ObjC propios; P3 propone el host Kotlin/Swift mínimo llamado desde Rust y medir Avalonia como plan B

**H335** — TENSION · CONCLUSIONES
Registrado por: Kimi
Respuestas que lo sostienen: Gemini, Mistral

> mismo diagnóstico (jetsam de iOS, proceso único de Android) pero P2 propone mantener Slint+wry con rotación de vistas en móvil como plan principal, mientras P5 concluye que si el teléfono exige diez simultáneos en pantalla solo sirve el caparazón nativo y Slint+wry queda para tableta/escritorio

**H336** — TENSION · CONCLUSIONES
Registrado por: Kimi
Respuestas que lo sostienen: Gemini, ChatGPT, Claude

> P2 usa la licencia de Slint (GPLv3/comercial/atribución) como argumento para considerar egui o Makepad; P1, P3 y el resto mantienen a Slint como favorito sin ponderar la licencia

**H337** — SINGULARIDAD · CONCLUSIONES
Registrado por: Kimi
Respuestas que lo sostienen: ChatGPT

> recomienda probar primero Native Shell (longbridge/nsh) antes que reescribir la interfaz, algo que ninguna otra respuesta considera

**H338** — SINGULARIDAD · HECHOS
Registrado por: Kimi
Respuestas que lo sostienen: ChatGPT

> datos de Native Shell 0.4.0 (4 nov 2026): Apache-2.0, informe del fabricante en Windows (~14 MB privada, ~60 MB working set, 15 MB ejecutable); la API de entrada con eventos de confianza solo funciona en macOS

**H339** — SINGULARIDAD · HECHOS
Registrado por: Kimi
Respuestas que lo sostienen: ChatGPT

> única que detalla el flujo de adjuntos: onShowFileChooser por vista en Android y WKUIDelegate personalizable desde iOS 18.4

**H340** — SINGULARIDAD · HECHOS
Registrado por: Kimi
Respuestas que lo sostienen: ChatGPT

> única que cita guibench (lumen-fx, 23 jul 2026, Arch/Weston) con tabla de arranque/RAM/binario por framework

**H341** — SINGULARIDAD · HECHOS
Registrado por: Kimi
Respuestas que lo sostienen: Claude

> única que inspeccionó el código fuente de wry 0.57.1: en Android un solo espacio de webview por Activity y set_bounds sin efecto; en iOS la subvista toma el tamaño del padre; conflicto WryActivity/AppCompatActivity con winit/GameActivity

**H342** — SINGULARIDAD · HECHOS
Registrado por: Kimi
Respuestas que lo sostienen: Claude

> única que menciona perfiles de WebView en Android (cada perfil obtiene su propio proceso renderizador) como vía de aislamiento

**H343** — SINGULARIDAD · HECHOS
Registrado por: Kimi
Respuestas que lo sostienen: Claude

> únicas cifras de Uno Platform: 300 ms/212 MB en Windows y 654 ms/301 MB en Android, medidas declaradas por el fabricante con Native AOT

**H344** — SINGULARIDAD · HECHOS
Registrado por: Kimi
Respuestas que lo sostienen: Mistral

> únicas cifras fechadas de versiones: Tauri 2.0.0 (2 oct 2024), línea 2.12 (26 sep 2026), tauri-runtime-wry 2.11.0 (30 abr 2026)

**H345** — SINGULARIDAD · HECHOS
Registrado por: Kimi
Respuestas que lo sostienen: Mistral

> única que menciona Tauri 3.0 en alfa (13 sep 2026) con runtime CEF solo para desktop, y Qt Bridge for Rust (27 oct 2026) sin Android/iOS

**H346** — SINGULARIDAD · HECHOS
Registrado por: Kimi
Respuestas que lo sostienen: Mistral

> única que cita una prueba de estrés de jetsam reportada: diez WKWebView activos terminados en ~1 segundo (blog Cross Mini App, 15 oct 2026)

**H347** — SINGULARIDAD · HECHOS
Registrado por: Kimi
Respuestas que lo sostienen: Mistral

> única que explicita el soporte oficial de Slint por plataforma: Android (Rust y C++) e iOS (solo Rust)

**H348** — SINGULARIDAD · HECHOS
Registrado por: Kimi
Respuestas que lo sostienen: Perplexity

> única que evalúa CEF embebido como opción (descartado: sin Android/iOS oficial, RAM/tamaño de Chromium)

**H349** — SINGULARIDAD · CONCLUSIONES
Registrado por: Kimi
Respuestas que lo sostienen: Gemini

> única que propone un diseño concreto con tamaños: trait MultiWebViewManager con ~350 líneas Kotlin y ~350 líneas Swift/ObjC exponiendo cinco funciones a Rust

**H350** — SINGULARIDAD · CONCLUSIONES
Registrado por: Kimi
Respuestas que lo sostienen: Gemini

> única que plantea la alternativa de un solo código con CMP aceptando 50-80 MB extra de RAM en escritorio por la JVM (Camino B)

**H354** — LIMITACION:OTRA
Registrado por: Kimi
Respuestas que lo sostienen: Grok

> la tabla de fuentes de P4 llegó vacía en el archivo (solo se conserva una URL en la lista final), así que no pude contrastar la mayoría de sus citas fechadas

## Limitaciones registradas por los operadores

**H66** — LIMITACION:AMBIGUEDAD
Registrado por: ChatGPT
Respuestas que lo sostienen: Claude, Grok, Mistral, Perplexity

> Las fechas de marzo y abril de 2026 para Tauri se refieren a hitos distintos, como el PR y la publicación de tauri-runtime-wry 2.11.0. El corpus tampoco usa de forma uniforme «ventana móvil» al describir pantallas completas y ventanas que coexisten en tabletas.

**H67** — LIMITACION:AMBIGUEDAD
Registrado por: ChatGPT
Respuestas que lo sostienen: Gemini, Claude, Kimi

> Los pasajes sobre licencia de Avalonia remiten a documentación y paquetes distintos. P3 enlaza un paquete 11.3.15, por lo que su referencia no resuelve por sí sola la discrepancia sobre el control de Avalonia 12.

**H68** — LIMITACION:AMBIGUEDAD
Registrado por: ChatGPT
Respuestas que lo sostienen: Gemini, Mistral, Kimi, Perplexity

> P2 y P5 describen egui/eframe como utilizable en iOS. P6 y P7 discuten su condición de plataforma oficialmente soportada. Los pasajes no establecen que las cuatro estén evaluando el mismo tipo de soporte.

**H69** — LIMITACION:AMBIGUEDAD
Registrado por: ChatGPT
Respuestas que lo sostienen: Gemini, Grok, Mistral, Kimi, Perplexity

> Las cifras de RAM y CPU para Tauri, egui y Slint mezclan proceso anfitrión, procesos auxiliares, reposo, picos y cargas distintas. El corpus no permite convertirlas en un orden de rendimiento para diez webviews.

**H70** — LIMITACION:OTRA
Registrado por: ChatGPT
Respuestas que lo sostienen: Gemini

> El adjunto conserva el aviso de que el elemento interactivo no se pudo cargar. La tabla textual está disponible, pero no pude determinar si el elemento añadía datos.

**H117** — LIMITACION:CORPUS
Registrado por: Mistral
Respuestas que lo sostienen: Grok

> la tabla de fuentes de P4 está incompleta o dañada (celdas vacías y un "estudio independiente 2026" sin nombre), lo que impide verificar a qué fuente corresponde cada cifra concreta de ese bloque

**H118** — LIMITACION:CORPUS
Registrado por: Mistral
Respuestas que lo sostienen: Kimi

> P6 intercala fragmentos de registro de uso de herramientas y títulos ajenos (peticiones de razonamiento interno) que no forman parte de la respuesta sustantiva; los traté como metadato y no como contenido analizable

**H255** — LIMITACION:CORPUS
Registrado por: Gemini
Respuestas que lo sostienen: ChatGPT, Claude

> El archivo adjunto contiene marcas de censura (■■■) que ocultan el nombre de un proveedor que restringe OAuth en navegadores embebidos en P1 y el nombre del sistema autor al inicio de P3, además de un fragmento truncado en el preámbulo de P3.

**H309** — LIMITACION:AMBIGUEDAD
Registrado por: Claude
Respuestas que lo sostienen: Grok, ChatGPT

> P4 cita “goldstrikearch Rust GUI ecosystem measured, 2026-09” y P1 cita un “ensayo público de macOS, del 15 ago 2026, una aplicación de cuatro paneles”; no puedo determinar con certeza si son la misma fuente o dos fuentes distintas porque las fechas y descripciones no coinciden exactamente, pero ambos citan goldstrikearch en sus listas de fuentes

**H354** — LIMITACION:OTRA
Registrado por: Kimi
Respuestas que lo sostienen: Grok

> la tabla de fuentes de P4 llegó vacía en el archivo (solo se conserva una URL en la lista final), así que no pude contrastar la mayoría de sus citas fechadas

## Condiciones de la ronda

La columna Código es la clave de los [P#] que aparecen en el texto del redactor y del integrador: es el código estable del sello de esta ronda.

| Código | Proveedor | Etiqueta de modelo | Caracteres de su respuesta | Fuentes citadas |
|---|---|---|---|---|
| P7 | Perplexity | (no observada) | 11996 | 6 |
| P4 | Grok | Fast | 10215 | 1 |
| P3 | Claude | Opus 5.5 Alto | 8245 | 44 |
| P5 | Mistral | (no observada) | 17404 | 54 |
| P2 | Gemini | Argon Alto | 49306 | 0 |
| P1 | ChatGPT | (no observada) | 13823 | 22 |
| P6 | Kimi | (no observada) | 13145 | 42 |

Conversación: d4fc7e58-7def-419a-9ab5-f500a5415376
Ronda: 0f31dbec-0cfb-42c4-96c7-794079c9993f

Integrador de esta ronda: DeepSeek

**Integridad de entrega:** no persistida en el registro (se informa en pantalla al pegar cada panel)
**Semilla de la ronda:** 7955480a-e5d9-4d02-b278-f052e54ae6b0

## Lo que este informe no dice

Este informe describe cómo se relacionan siete respuestas entre sí. No determina cuál es correcta. La verificación mecánica comprueba que una fuente existe y coincide, no que sostenga la afirmación. El informe del integrador es una afirmación de un modelo con su procedencia registrada, no un resultado verificado. Las limitaciones completas del instrumento están en docs/LIMITACIONES.md.